Cluster Computing (2025) 28:515
https://doi.org/10.1007/s10586-025-05171-w

Building robust lightweight network intrusion detection: a multi-pronged approach with DSSTE and deep learning for securing edge-enabled industrial IoT applications

Mohemmed Yousuf Rahamathulla[1] · Mangayarkarasi Ramaiah[1]

Received: 4 September 2024 / Revised: 23 December 2024 / Accepted: 6 February 2025 / Published online: 19 August 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Industrial automation increasingly relies on interoperable Industrial Internet of Things (IIoT) devices, but ensuring secure communication across diverse protocols remains a challenge. Edge computing offers a solution by accelerating data analysis closer to devices. However, this introduces new vulnerabilities requiring vigilant network monitoring. To address this, we propose a comprehensive Network Intrusion Detection System (NIDS) framework tailored for edge-based IIoT environments. Our framework includes data preprocessing, sampling, feature selection, and a deep learning-based anomaly detection model. To alleviate the data sample crisis, we employed the Difficult Set Sampling for Training Efficiency (DSSTE) algorithm. A hybrid feature selection approach, leveraging the strengths of RF, XGBoost-based shuffling, filter, wrapper, and embedded methods, was used to identify critical features. Experimental results demonstrate the presented NIDS's superior accuracy (99.9%) and low false alarm rate (0.04%) in detecting various attacks, even with a reduced feature set. The frameworks efficiency and effectiveness highlight its potential for enhancing the security of edge-based IIoT networks.

Keywords Deep learning · DSSTE · Edge-IIoT · Feature selection · NIDS

1 Introduction

Security threats have increased as a result of the extraordinary growth in the number of devices connected to the Internet. The explosion of interconnected Internet of Things (IoT) devices, along with the vast amount of data they generate, has created a tempting target for cybercriminals. To protect this sensitive data and prevent attacks, robust security measures are crucial. This is especially true for the IIoT, a critical part of the IoT ecosystem that connects industrial machinery and sensors. Cloud-based processing can introduce delays due to data traveling long distances. Edge computing brings data processing closer to the source (IoT devices or sensors) in IIoT environments. This reduces latency, allowing for near real-time data analysis and faster decision-making. Distributing processing and storage across the network can make it harder for attackers to target a single point of vulnerability. Additionally, edge devices can perform preliminary security checks on data before sending it to the cloud, reducing the risk of compromised information.

While edge computing offers benefits like faster data processing and lower latency by bringing data storage and management closer to IIoT devices, it also introduces new security concerns. Security threats that once resided in the cloud are now distributed across the various layers of edge architectures [1] may compromises the security aspects. Moreover, edge devices are physically accessible, increasing the risk of tampering and unauthorized data access during the data sharing. Even after successful authentication, edge devices remain vulnerable to compromise by insiders. These insiders can exploit the access privileges granted to the devices for legitimate internal network activities to perform unauthorized actions. This

✉ Mangayarkarasi Ramaiah
rmangayarkarasi@vit.ac.in

Mohemmed Yousuf Rahamathulla
mohemmedyousuf.r2020@vitstudent.ac.in

[1] School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, Tamil Nadu, India

type of attack, also known as an insider attack or unauthorized access attempt, targets the edge layer or cloud layer [2]. Attackers often leverage compromised smart devices, turning them into "bots" under their control, to achieve their malicious goals [3]. Firewalls, traditionally used for defense, are ineffective against insider attacks originating within the network. Edge computing is susceptible to various attacks, including network attacks [Man-in-the-Middle (MITM), Distributed Denial-of-Service (DDoS)], device attacks (device hijacking, malware injection), and data security attacks (side-channel attacks, data leakage through MITM) [4]. While security measures like homomorphic encryption, public key cryptography, and access control are employed, there's a constant need for even more effective solutions to safeguard edge enabled computing environment. The field of cybersecurity is witnessing a surge in interest towards NIDS for Industrial Internet of Things (IIoT) [5] applications. These systems aim to bolster information security by pinpointing a wide range of attacks that could potentially disrupt the normal operations of IIoT, particularly those leveraging distributed edge computing.

IIoT systems require strong intrusion detection systems to ensure precise results in automation without compromising the security aspects [6, 7]. Security risks may target to compromise a network's service's availability, confidentiality, and integrity. In terms of combating security risks and identifying attacks via incoming network traffic, intrusion detection systems (IDSs) are in the lead [8]. Security professionals work tirelessly to create efficient strategies to counter both known and unknown threats. Intrusion detection systems (IDS) are a critical line of defense in information security. They act as vigilant guards, constantly monitoring systems for unusual activity that might indicate a security breach [9]. NIDS faces several challenges on with massive volume of data. IDSs filter incoming traffic and categorize incoming packets as either legitimate or malicious. Recent research [10, 11] demonstrates that ML based algorithms can filter out irrelevant network traffic data, minimizing false positives (flagging normal activity as malicious). ML based attack detection built upon the Several network features. There are many features present in the incoming packets. The effectiveness and cost of IDSs are adversely impacted by including every features [12] in the detection process. Building strong intrusion detection systems (IDSs) requires the robust feature selection approach to enhance the anomaly detection [13]. Low accuracy and a high false negative rate are the outcomes of poorly chosen features.

Detection of IoT [14] bot detection is enhanced with better precision thorough the selection minimal number of features. Machine learning can automate the process of selecting the most relevant features from network traffic data. Effective IDSs are primarily evaluated based on two factors: a high detection rate and a low false alarm rate [15]. NIDSs based on deep learning have proven to be more accurate and superior, however, they are unable to identify cyber-attacks with lower traffic because of class imbalances in the benchmark datasets. Data imbalances against different class labels pose a technical hindrance in the cyber-attack detection process in the form introducing over-fitting and under-fitting issues. To mitigate the technical challenges incurred through the data imbalance a sampling enabled ML models were trained and tested for cyber-attacks is presented by [16]. In the literature [17], to address class imbalance in cybersecurity datasets, researchers employed various resampling techniques on benchmark collections. These techniques included random under-sampling, random oversampling, combinations of both, and more advanced methods like SMOTE (Synthetic Minority Oversampling Technique) and ADASYN (Adaptive Synthetic Sampling Method). The datasets used for evaluation were KDD99, UNSW-NB15, UNSW-NB17, and UNSW-NB18. But, introducing data sampling technique should consider various aspects, Oversampling leads to longer training times due to the creation of additional data points. Conversely, under-sampling reduces training time by removing data. When dealing with datasets where attack traffic is rare (highly imbalanced data), techniques like oversampling and under-sampling can be valuable tools for improving attack detection. For moderately imbalanced data, resampling techniques have minimal impact on overall performance. Oversampling, in particular, led to a higher detection rate for minority classes (attacks) within the resampled datasets. Hence, choosing appropriate data sampling is essential to improve the network intrusion detection. One of the key challenges in designing cyber-attack detection systems is finding suitable datasets.

While numerous studies have explored the application of deep learning techniques to intrusion detection in IoT networks, many existing approaches suffer from limitations such as high computational costs, scalability issues, and sensitivity to data quality. For instance, while the work presented in [18] is good enough only to detect the DDoS attacks and might not be effective against the broader range of cyberattacks expected in edge-enabled IIoT environments. Therefore, for this specific context, training models on network traffic attack vectors simulated within an IIoT environment would be more suitable. Hence, this paper proposes a novel approach that leverages a Multi-Pronged approach to enhance the accuracy, efficiency, and robustness of intrusion detection systems. To make it be more comprehensive, we implemented various techniques, including DSSTE, a hybrid feature selection, to optimize the computational complexity. Sampling deficiency issues

has been addressed through DSSTE (Difficult Set Sampling for Training Efficiency) which focuses on the most challenging samples within a dataset—those that are difficult for the model to classify. By selectively training on these samples, DSSTE enhances model performance and generalization. Concentrating on difficult instances helps the model make more accurate predictions, prevents overfitting to easy-to-classify samples, and accelerates the training process by reducing the data volume used.

In addition, this study deploys a novel two-stage feature selection process that combines the strengths of filter, wrapper, and embedded methods to effectively reduce dimensionality. In the first phase, feature shuffling using RF and XGBoost is carried out to remove irrelevant features. In the second phase, Rank-Based Feature Selection prioritizes features by their importance, selecting a subset of the most informative ones. This refined feature set results in simpler, faster-to-train, and more efficient models. These aspects contribute to a lightweight model, making it suitable for resource-constrained IoT devices. The framework's optimized components enable real-time processing of network traffic, ensuring timely intrusion detection. Furthermore, the framework can be adapted to handle large-scale IIoT networks by scaling the model and utilizing distributed computing techniques.

Major highlights of the presented work is as follows:

1.1 Novel contribution

This study intends to tackle the problem of class imbalances to increase the NIDS's ability to identify minority classes. The following constitutes the study's main contribution:

- We conduct a deep analysis of the EdgeIIoTset dataset [15] to understand the wide range of threats and their characteristics.
- This study proposes a DSSTE based hybrid sampling technique to tackle the challenges of imbalanced data in EdgeIIoT datasets.
- A two-phase feature selection process is employed. In the first phase, feature shuffling is utilized. The second phase leverages a hybrid approach, combining filter, wrapper, and embedded methods for rank-based feature selection.
- The performance of the proposed feature selection method was assessed using a variety of deep learning models, including LSTM, GAN, GNN, DQN, and DNN.
- Finally, the obtained results are benchmarked against existing NIDS frameworks.

The paper is organized as follows: Sect. 2: Reviews related work in the field. Section 3: Highlights the Proposed Methodology. Section 4 describes the suggested classification techniques. Section 5: Covers the description of the dataset, data preprocessing, feature engineering, feature selection techniques, experimental setup and the performance metrics used for evaluation. Section 6: Discusses the findings of the study and presents any relevant debates or controversies. Section 7: Concludes the paper by summarizing the key takeaways and outlining potential areas for future research.

2 Related works

The growing threat of network breaches has spurred significant research into Network Intrusion Detection Systems (NIDS) powered by deep learning. While defense experts have developed advanced NIDS in recent years, attackers are constantly devising new methods to bypass them. To effectively address the security challenges incurred due to the deploy to the inclusion of by the Internet of Things (IoT), a detailed understanding of existing NIDS methodologies is crucial. This section delves into a critical examination of established NIDS approaches presented in various academic publications.

One critical challenge facing machine learning-based attack classification systems is data imbalance in training samples. Network traffic simulations inherently struggle to achieve balanced representation across all attack types (output class labels). Popoola et al. [19] investigated this issue using the Bot-IoT dataset. Their study explored the effectiveness of the SMOTE oversampling technique along with Deep Recurrent Neural Networks (DRNN) for addressing the under representation of minority attack classes. While their findings show improved accuracy and F1-score with SMOTE-DRNN compared to DRNN alone, the study also highlights limitations. DRNN appears susceptible to class imbalance during training, potentially hindering its overall performance.

To address data imbalance, Tao Wu et al. [20] built a Random Forest based intruder detection upon the samples generated using SMOTE. Their approach demonstrates the results obtained by combining SMOTE with KNN (K-Nearest Neighbors). The high training accuracy (99.72%) and relatively lower testing accuracy (78.47%) in your experiment suggest a potential issue of overfitting. Bagui and Li [17] employed several re-sampling methods, such as the adaptive synthetic minority technique, by integrating under-sampling with SMOTE and then random over-sampling. They derived several key insights from their model: Firstly, utilizing only oversampling techniques can lead to an increase in training time. Secondly, in scenarios where the data is significantly imbalanced and includes numerous minority classes, the application of both kinds sampling

techniques substantially boosts recall. Lastly, if the dataset is only mildly imbalanced and the minority class is small, resampling may not be necessary, as it does not significantly alter the results. To tackle issues related to unbalanced data, Tian et al. [21] utilized the SMOTE technique. They employed a Convolutional Neural Network (CNN) to analyze the raw traffic data, which was represented as images. Their results demonstrated a performance superior to that of other machine learning methods.

Another area of focus for researchers is feature selection. Studies have investigated feature selection techniques to over- come the curse of dimensionality and improve model performance and generalization. Both feature extraction and feature selection aim to reduce dimensionality, but they do so differently. Feature extraction deduces the subset to facilitate to infer more insights, while feature selection eliminates redundant or irrelevant features. Feature selection plays a critical role in Intrusion Detection Systems (IDSs). It helps minimize the complexity of the data size by removing unwanted feature column upon various metrics.

A genetic algorithm (GA) combined with a mutual information (MI)-based technique was proposed as a feature selection method in a study by [22]. This method achieved promising results on the ADFA-LD dataset, which contains 49 features. Incorrect feature selection can result in low accuracy and a high false negative rate. To address this issue and select useful features, the CorrACC technique was proposed [23]. A recent wrapper-based approach called CorrACC achieved a promising accuracy of 95.0% using four classifiers on the BoT-IoT dataset. A study by [24] proposed a method based on multi-objective based feature selection achieving a high accuracy (99.9%) in detecting DDoS attacks on the CICIDS-2017 dataset, while selecting fewer features compared to other approaches. While feature selection can improve efficiency, the multi-objective optimization process itself might be computationally expensive. Understanding the rationale behind the selected features can be challenging, especially with complex optimization algorithms. The effectiveness of the method [22] suggests a good balance between accuracy and the resources required for feature selection. Emphasizing this efficiency concept, the authors in [25] proposed a process flow for filter-based feature selection. In pursuit of a more efficient selection process, their approach leverages five distinct transformation techniques. The performance of the method might vary on different datasets, especially those with different characteristics.

Ren et al. [26] proposed a hybrid data optimization method named DO-IDS, which includes two main components: data sampling and feature selection. This method forms the basis of an efficient Intrusion Detection System (IDS). The approach utilizes Genetic Algorithms (GA) to optimize the sample ratio for training data. A RF classifier is used to assess different training sets and choose the one that performs best. Additionally, Isolation Forest (iForest) helps eliminate outliers during the data sampling process. GA and RF were also utilized to select the optimal feature subset. This model demonstrated superior performance in detecting unusual anomalous behavior compared to other algorithms. The performance of the approach might be sensitive to the choice of GA parameters and the RF classifier's hyperparameters. Understanding the rationale behind the selected features and sample ratios might be challenging. Their process flow method achieved higher efficiency and accuracy in selecting a more relevant set of features compared to existing feature selection and process flow methods. Gain ratio (GR) and information gain (IG) were applied to specific subsets of features. Following the union operation, the top 50% of features were selected based on their Information Gain (IG) and Gini Ratio (GR) scores. This combined approach ensured that the chosen features offered the most informative power for attack detection.

Nimbalkar and Kshirsagar [27] recommended using the JRip classifier, and after applying the selected features and performing necessary validation, this approach achieved an impressive 99.9% accuracy on both the BoT-IoT and KDDCup99 datasets. Bio-inspired optimization techniques have been employed to select dominant features. Elmasry et al. [28] uses the Particle Swarm Optimization (PSO) method to select the significant features and derive the hyper-parameters for the deep learning models. The candidate study provides a thorough empirical analysis comparing ensembles of Long Short-Term Memory Recurrent Neural Networks (LSTM-RNN), Deep Belief Networks (DBN), and Deep Neural Networks (DNN) for intrusion detection. Notably, these models are pre-trained using a double PSO-based metaheuristic approach. This approach automates the process of finding the optimal hyperparameter settings for each model, ultimately aiming to achieve the highest accuracy even on smaller datasets. Additionally, Devan and Khare [29] proposed the XGBoost-DNN approach, which uses XGBoost for feature selection followed by DNN for classifying network intrusions. The XGBoost-DNN model comprises three phases: feature selection, normalization, and categorization. Network intrusions were classified using the Softmax classifier. This approach outperformed previous rudimentary methods applied to the same dataset.

A novel hybrid CNN-LSTM model is proposed in [30] for intrusion detection in IIoT networks. The model effectively classifies network traffic, achieving an accuracy of 97.85% for binary classification and 97.14% for multiclass classification. By leveraging only transport and network layer features, the model protects privacy while

maintaining high performance. The research in [31] presents a novel deep learning-based intrusion detection system (IDS) specifically designed for edge-based smart agriculture in extreme environments. The system utilizes a hybrid approach combining bidirectional gated recurrent units (BiGRU), long-short-term memory (LSTM), and a softmax classifier to effectively detect attacks at the network edge. To improve training efficiency, the IDS incorporates truncated backpropagation through time (TBPTT) for handling lengthy network data sequences. A comprehensive evaluation was conducted using three publicly available datasets: CIC-IDS2018, ToN-IoT, and Edge-IIoTset. The proposed IDS demonstrated superior performance compared to existing techniques, achieving high accuracy rates (99.82, 99.55, and 98.32%) and low false positive rates (0.0369, 0.0031, and 0.0426%) across the datasets.

The study in [32] proposes three deep learning-based intrusion detection models for IIoT networks: CNN, LSTM, and a hybrid CNN + LSTM architecture. The models were evaluated on UNSW-NB15 and X-IIoTID datasets, focusing on binary and multi-class classification. The hybrid CNN + LSTM model consistently outperformed the individual CNN and LSTM models, achieving the highest accuracy rates. Specifically, it reached 93.21% for binary classification and 92.9% for multi-class classification on UNSW-NB15, and 99.84 and 99.80%, respectively, on X-IIoTID.

The study in [33] presents a comprehensive framework to enhance the security of 5G-enabled IoT networks. A hierarchical architecture is proposed to address security challenges at multiple layers, from the network edge to the cloud. To mitigate the threat of False Data Injection Attacks (FDIA) and Distributed Denial of Service (DDoS) attacks, a stochastic Markov model is employed to monitor network device behavior and identify anomalies. The proposed framework effectively detects and mitigates these attacks, demonstrating promising results in simulations. However, scalability and computational overhead remain potential limitations, particularly in resource-constrained edge environments. A novel framework, FedIoT is proposed in [34] to address security and privacy concerns in federated learning for IoT systems. FedIoT incorporates XAI techniques to enhance model interpretability and identify potential attacks. A blockchain-based reputation system is proposed to ensure the trustworthiness of participating devices in the federated learning process. The framework demonstrates effective detection of malicious activities in IoT networks. Rigorous evaluation using the UNSW-NB15 dataset confirms the superior performance of FedIoT compared to centralized approaches. The additional overhead of Explainable AI (XAI) techniques and blockchain-based mechanisms may impact the efficiency of the system, especially in resource-constrained IoT devices. Handling data heterogeneity across different devices can be challenging and may require careful feature engineering and model adaptation.The communication overhead of federated learning, especially in large-scale IoT deployments, can be significant.

The research in [35] introduced a novel Deep Reinforcement Learning (DRL) based intrusion detection system for 5G IoT networks. The system leverages a distributed Q-learning algorithm to enable adaptive learning and decision-making among IoT devices. A reputation-based bandwidth allocation mechanism is also proposed to optimize resource utilization. The proposed system demonstrates superior performance compared to state-of-the-art techniques. However, the performance of the system is heavily dependent on the quality and quantity of training data. The computational overhead of DRL algorithms, particularly on resource-constrained edge devices, remains a significant challenge. Recent research in [36] has explored the application of federated learning (FL) along with homomorphic encryption (HE) to enhance the security of IoT networks. This approach enables collaborative learning among devices while preserving data privacy by encrypting data before sharing it. Secure aggregation techniques, such as those based on multi-party computation, can be used to aggregate model updates without revealing individual device data. However, challenges such as secure aggregation and communication efficiency remain. Homomorphic encryption can be computationally intensive, limiting its practical application. Additionally, secure aggregation can increase communication overhead, especially in large-scale networks.

To summarize the comprehensive analysis, Table 1 shows the state of the art NIDS intent to mitigate the cyber-attacks anticipated in the IoT Environment. Among the works tabulated in Table 1. CICIDS2018 [37], CIDDS-001 [38] ADFA-LD, KDD [22], CICIDS-2017, ISCX-IDS 2012 [25], UNSW-NB15 [26] and NSL-KDD [28] are not compiled in IoT or IIoT environment. Hence, the ML models built upon then may not be appropriate to detect the cyber-attacks in IoT/IIoT environment. Therefore, upon analyzing the diverse NIDS frameworks outlined in Table 1, the identified research topic requires enhancements in addressing data imbalance and implementing robust feature selection methods, along with optimizing deep learning models. Moreover, for NIDS tailored to Edge-enabled IIoT environments, scalability is crucial, necessitating reduced latency and increased throughput. Implementing these improvements calls for the adoption of a lightweight yet robust NIDS capable of learning and adapting to new cyber-attacks anticipated in the IIoT environment.

Table 1 Comparative analysis of state-of-the-art technologies for intrusion detection in IIoT networks: dataset, leveraged techniques, and merits

References	Dataset	Classifiers	Feature selection	Data balancing	Merits	Limitations
[19]	Bot-IoT	DRNN	NA	SMOTE	High Accuracy and Low FPR	Computationally intensive, prolonged training time
[22]	ADFA-LD, KDD	ML Classifiers	GA and MI	NA	High Accuracy	Imbalanced Classes
[23]	BoT-IoT	ML Classifiers	CorrACC	NA	High Accuracy	Training affected by Sensitive data
[25]	CIC-IDS2017, ISCX-IDS 2012	DNN	Filter based feature selection	SMOTE + ENN	Low execution time, reduced overfitting	Lacks performance across classes
[26]	UNSW-NB15	RF	GA and RF	GA	Realistic and labeled data	Noisy, imbalanced classes
[27]	BoT-IoT, KDDCup 99	JRip	IG, GR	NA	99.9 and 99.5% Accuracy	Limited to DoS, DDoS attacks
[28]	NSL-KDD	LSTM-RNN, DNN	PSO	NA	Low execution time, reduced overfitting	Parameter sensitive, Computationally complex
[29]	NSL-KDD	DNN—softmax	XGBoost	NA	Efficient and adaptive learning rate	Generalization, Memory consumption
[30]	EdgeIIoT	CNN + LSTM	NA	NA	High Accuracy, Privacy Preservation	Computational complexity, Interpretability issues
[31]	CICIDS2018, ToN-IoT, EdgeIIoT	BiGRU-LSTM	NA	NA	High Accuracy and low FPR	Biased outcome due to class imbalance
[32]	X-IIoTID	CNN + LSTM	NA	NA	Robust, High Accuracy	Data imbalance, Interpretability, Computational Complexity
[33]	Simulated Dataset	Stochastic Markov model	NA	NA	Stable detection rate	Computational overhead, scalability
[34]	UNSW-NB15	Blockchain-FedIoT	NA	NA	High Accuracy and F1-score	Communication and computation overhaead,
[35]	NSL-KDD	DRL	NA	NA	Optimal resource allocation, Better F1-scores	Data Sensitive, Computational overhead
[36]	UNSW-NB15	Homomorphic Encryption-FL	NA	NA	Data Integrity and transparency	Data and device heterogeneity
[37]	CICIDS-2018	ML Classifiers	Univariate selection	SMOTE	Reduction of noise, multi-collinearity	Limited scope, potential information loss
[38]	CIDDS-001	CNN + LSTM	NA	SMOTE + TOMEK	Reduced overfitting, improved model performance	Computational complexity
[39]	CIC-DDoS2019, EdgeIIoT	ML Classifiers	RoC based Hybrid model	SMOTE + TOMEK	Information preservation, Noise sensitivity	Potential Overfitting, persisting class imbalance

3 Proposed work

The presented Edge based network intrusion system encompasses various steps, data preprocessing, data-sampling, feature selection, deep learning based model training, as illustrated in Fig. 1. Data preprocessing is a crucial step that cleans the dataset by removing features with missing values and high variance. This process helps create a high-quality dataset suitable for further analysis. To tackle the class imbalance issue, a hybrid data sampling strategy Difficult Set Sampling Technique (DSSTE) is employed. DSSTE facilitates the classifier performance on

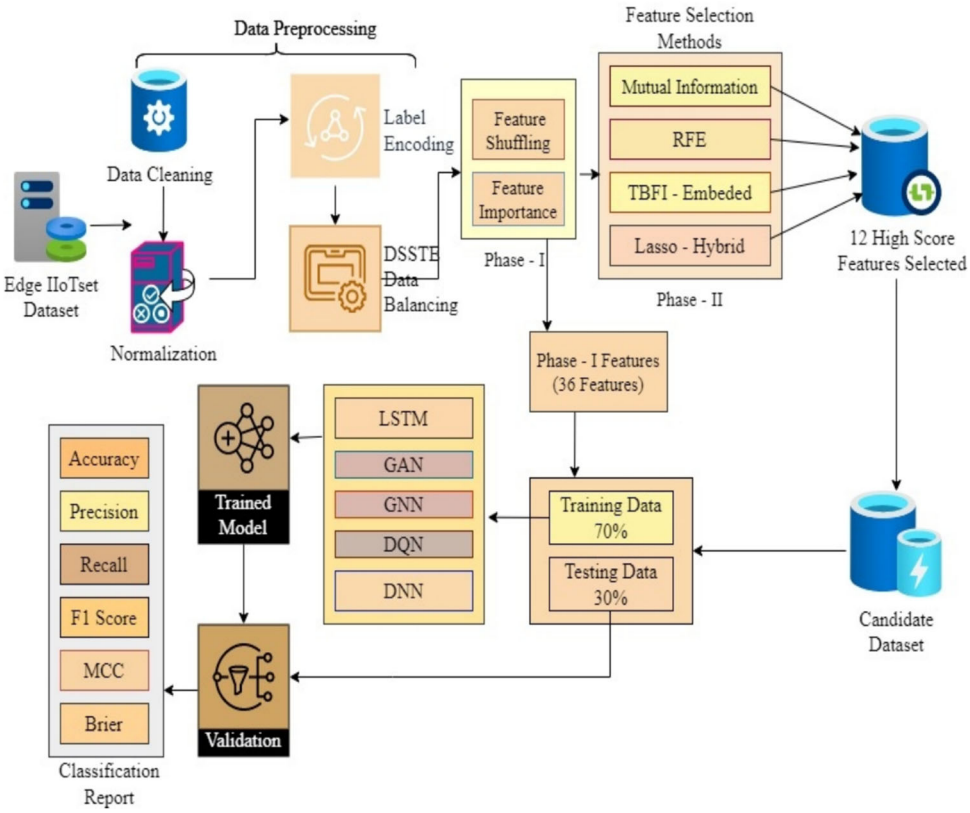


Fig. 1 Proposed model with feature selection

unbalanced datasets by partially correcting the class distribution. Dominant feature selection is carried out using feature shuffling and feature ranking. Phase I of the feature selection process employs a lightweight permutation-based feature importance method to identify significant features. This approach utilizes the Random Forest (RF) algorithm to reduce the dataset from 48 features to 36. Subsequently, feature importance computation is performed again using the XGBoost (XGB) algorithm to identify the top 20 features. The results from Phase I are then fed into the second phase of the feature selection process. Phase-II of the feature selection process entails applying various feature selection techniques such as filter-based, wrapper-based, embedded, hybrid techniques to the top 20 features obtained as a result from Phase-I. This process involved ranking the features and evaluating how frequently each one was selected by various feature selection methods. A candidate dataset was then created, including only features with a feature score of at least 3. This step further refined the candidate feature set, reducing it to 12 features. The first step in classification is dividing the candidate dataset into training and testing datasets. Then, the training dataset is subjected to a variety of classification techniques to produce models, and the test dataset is provided for various deep learning model development.

4 EdgeIIoT cyberattack detection models

This section details the inner workings of cyberattack detection models designed for the candidate task. Real-world data frequently exhibits complex, non-linear relationships. Cyberattack detection models, with their stacked layers, are particularly adept at capturing these intricate patterns. This capability is especially valuable for anomaly detection, as anomalies often deviate from the usual, linear behavior.

4.1 Deep learning based detection models

Traditional cyberattack detection methods often rely on predefined rules and signatures, which can be ineffective against new and sophisticated attacks. Deep learning has emerged as a powerful tool for enhancing cyberattack detection models due to its ability to automatically learn complex patterns from large amounts of data. A selection of deep learning-based cyberattack detection models is discussed below.

4.1.1 LSTM

Using LSTM in intrusion detection systems (IDS) is an innovative approach that leverages the network's ability to model temporal dependencies in data. LSTM can learn

hierarchical representations of network traffic features. Instead of relying solely on handcrafted features, LSTM can automatically extract relevant features from raw network data, potentially capturing subtle patterns that might be missed by traditional IDS approaches. LSTM can be trained on a dataset containing normal network traffic behavior. During deployment, the model can detect deviations from the learned normal behavior, flagging them as anomalies or potential intrusion attempts. The ability of LSTM to capture long-term dependencies allows it to detect complex and stealthy attacks that evolve over time. LSTM models can process network data in real-time, making them suitable for deployment in online intrusion detection systems. By continuously analyzing incoming network traffic, LSTM-based IDS can promptly detect and respond to emerging threats. LSTM models excel in learning the dynamism in the networks. This allows them to continuously refine their understanding of normal network traffic patterns, ultimately maintaining high accuracy in anomaly detection even in dynamic environments. Table 2 outlines the hyperparameter settings used for the LSTM model.

4.1.2 GAN

Intrusion detection systems (IDS) can benefit greatly from a novel approach that leverages Generative Adversarial Networks (GANs). Synthetic network traffic data generated by GANs is remarkably realistic, closely mimicking the patterns observed in real-world networks. This capability offers several advantages for IDS development and improvement. By training a GAN on a dataset of normal network traffic, it can learn the underlying distribution of benign traffic patterns. The generated synthetic data can then be used to augment the training dataset for training traditional IDS models, thereby improving their robustness and generalization ability GANs can be used to craft adversarial examples specifically designed to bypass IDS. By training IDS models on adversary generated samples, they can become more robust to attacks and better able to detect sophisticated evasion techniques employed by attackers. GANs can adapt to changes in network traffic patterns over time. As the network environment evolves, the GAN can continuously learn and update its understanding of normal traffic behavior, thereby improving the accuracy of anomaly detection in real-time. Table 3 outlines the hyper-parameter settings used for the GAN model.

4.1.3 GNN

Using Graph Neural Networks (GNNs) in intrusion detection systems (IDS) is an emerging approach that leverages the graph structure inherent in network data. Network data, such as communication between hosts or devices, can be naturally represented as graphs, where nodes represent network entities (e.g., computers, routers) and edges represent connections or interactions between them. GNNs can directly operate on this graph structure, enabling them to capture complex relationships and dependencies between network entities. In an IDS, each node in the network graph can represent a host, device, or network entity. GNNs can perform node classification tasks, where each node is classified as either benign or malicious based on its features and its relationships with neighboring nodes. This allows GNNs to detect anomalous behavior at the level of individual network entities. Instead of classifying individual nodes, GNNs can classify entire subgraphs or graphs representing different segments of the network. This approach is particularly useful for detecting coordinated attacks or network-wide anomalies that involve multiple entities. GNNs can learn to identify patterns of malicious behavior that span across the network graph. GNNs can be designed to be robust against adversarial attacks, where attackers manipulate the network data to evade detection. By incorporating techniques such as adversarial training or robust optimization, GNN-based IDS can defend against attacks aimed at exploiting vulnerabilities in the detection model. GNNs can be extended to incorporate temporal information, allowing them to model how network behavior evolves over time. This enables GNN-based IDS to detect temporal patterns of malicious activity and identify ongoing or evolving threats in real-time. Table 4

Table 2 Hyper-parameter setting for LSTM Model

Hyperparameters	Default Value
Learning rate	0.1
Number of layers	5
Number of units/layer	64
Dropout rate	0.2
Activation function	ReLU

Table 3 Hyper-parameter setting for GAN Model

Hyperparameters	Default value
Learning rate	0.1
Batch size	64
Noise distribution	Gaussian
Number of epochs	200

Table 4 Hyper-parameter setting for the GNN Model

Hyperparameters	Default value
Learning rate	0.1
Number of layers	5
Number of units/layer	64
Aggregate function	Sum

outlines the hyperparameter settings used for the GNN model.

4.1.4 DQN

Applying Deep Q-Networks (DQN) in intrusion detection systems (IDS) is a novel approach that can offer several advantages. DQN operates within a reinforcement learning framework, where the IDS agent interacts with its environment (the network traffic) and learns to take actions (detect or classify network traffic) to maximize a cumulative reward (detecting intrusions while minimizing false positives). DQN requires a state representation of the environment. In the context of IDS, this could include features extracted from net-work packets, flow data, or other relevant information about the network's current state. In an IDS, the reward function might assign positive rewards for correctly detecting intrusions and negative rewards for false alarms. Balancing these rewards is important to ensure the IDS prioritizes accurate detection while minimizing false positives. Table 5 outlines the hyper-parameter settings used for the DQN model.

4.1.5 DNN

Deep Neural Networks (DNNs) is a kind of ANN with multiple hidden layers stacked between the input and output layers. These hidden layers permit DNNs to infer intricate patterns and relationships among them. DNNs have emerged as powerful tools for diverse applications, including image recognition, IDS, and natural language processing. Their ability to learn intricate patterns from network traffic data makes them particularly promising for IDS applications. By training on large datasets, DNNs can be equipped to classify different attack types, such as denial-of-service attacks or data breaches. This capability empowers security personnel to prioritize and respond to threats more effectively, ultimately enhancing overall network security. Table 6 outlines the hyper-parameter settings used for the DNN model.

Table 5 Hyper-parameter setting for the DQN Model

Hyperparameters	Default value
Learning rate	0.1
Exploration–exploitation trade-off, ε	0.2
Replay buffer size (MB)	1000

4.2 Challenges and limitations

This section highlights the inherent challenges associated with deep learning-enabled anomaly detection models. Specifically, their computational intensity can pose challenges, especially for resource-constrained edge devices. To address this, we carefully optimized various hyperparameters, including the learning rate, network architecture (number of layers and units), dropout rate, batch size, number of epochs, aggregate function, regularization techniques, replay buffer size, exploration–exploitation trade-off, and activation functions. This optimization process aimed to strike a balance between model complexity and performance, preventing overfitting while ensuring efficient learning. By tuning these parameters, we were able to improve the model's computational efficiency and enhance its ability to detect intrusions accurately. Though the study presents potentially interesting findings regarding deep learning models for intrusion detection, several methodological and reporting limitations significantly weaken its conclusions. The study emphasizes the low False Alarm Rate (FAR) achieved by the GAN model. However, the significance of a low FAR depends on the costs associated with false positives, such as wasted resources on investigating non-existent threats. The cost analysis is missing. In conclusion, while the study's exploration of deep learning for intrusion detection is a valuable endeavor, the aforementioned limitation concerning large dataset handling, feature extraction, and most importantly, the lack of model interpretation may significantly limit the strength and reliability of its conclusions.

Table 6 Hyper-parameter setting for the DNN Model

Hypermarameters	Default value
Number of layers	5
Number of units/layer	64
Activation function	ReLU
Regularization	L2
Dropout	0.2

5 Evaluation and analysis

5.1 Dataset description

The presented IIoT-NIDS leverages the EdgeIIoT dataset [15]. The dataset includes samples from five attack classes. Overall, it contains 20,953,689 samples with 63 features. Table 7 provides a breakdown of the EdgeIIoT dataset, listing the different attack classes and the corresponding number of records for each class. Following the recommendations in reference [40], the dataset was purged of low-correlated attributes, resulting in 48 features and the same is depicted in Table 8. Out of which five percent of the records were taken from each of the classes for training and validation of the learning models. Table 9 details the five percent of the EdgeIIoT dataset class distribution. Figure 2 visualizes the distribution of Attack Classes in the Dataset.

To evaluate the proposed solution's resilience, the candidate experiment is also tested on the CIC-DDoS2019 dataset [41]. The utilized dataset is a compilation of three sources: CSE-CIC-IDS2018-AWS, CICIDS2017, and CICDoS (2016). It comprises 2,830,743 samples, each characterized by 84 features, and categorized into two classes: "Benign" and "Attack". 15 highly correlated features were removed from the CIC-DDoS2019 dataset using correlation analysis, resulting in a dataset with 68 features. Table 10 shows the class distribution used in the experiment, while Fig. 3 visually represents the sample distribution across these classes. Table 11 present the extracted features from the EdgeIIoT and CIC-DDoS2019 datasets, respectively.

5.2 Data preprocessing

5.2.1 Data cleaning

Data cleaning is a crucial step before moving on to further data processing because it removes unwanted observations, redundancies, and noise from the dataset. These factors can lead to errors. Inaccurate information can come from many different places. The caliber of the training data may have a significant impact on a classifier's accuracy. A vital component of deep learning is data cleaning. Reliable results and precise model generation are produced by clean datasets. Apart from the best-organized and synthetic datasets, the majority of recorded data probably contains some noise.

Table 7 Distribution of classes within the EdgeIIoT dataset

Distribution name	Records
Normal—C1	11,223,940
DDoS—C2	8,365,162
Injection—C3	104,752
Malware—C4	1,089,172
Scanning—C5	169,434
MITM—C6	1229
Total No. of observations	20,953,689
Total No. of features	63
Total No. of attack samples	9,729,749
Total No. of benign samples	11,223,940

5.2.2 Data normalization

Z-score normalization has been considered. Equation (1) is used to normalize the data and transform it to fix into the appropriate range of values between 0 and 1.

$$N = \frac{S_s - \emptyset}{S_d} \tag{1}$$

where

- $\varnothing$ represents all the data points.
- S_s is the standard score.
- S_d is the standard deviation of the data points presented.

5.2.3 Label encoding

Label encoding converts categorical variables into numerical labels. Each unique category value is assigned an integer value. The different class of attack vectors were converted into numerical values like 0, 1, and 2. Categorical columns are changed into numerical values using label encoding helps make the data compatible with deep learning algorithms. By carefully applying label encoding to categorical features in an intrusion detection dataset, you can effectively prepare your data for training various deep learning models, improving both the performance and efficiency of your intrusion detection system.

5.2.4 Hybrid model for class imbalance

Data are further examined for balance or imbalance following pre-processing. One disadvantage of using intrusion detection datasets is that their extreme imbalance has the potential to increase model accuracy and thus impact the system detection rate. Incorrect handling of imbalanced classes is necessary because they can skew the model's performance by sharply raising the false alarm rate and accuracy. They can also have an impact on how classes are assigned. The suggested model makes use of resampling techniques to address the highly imbalanced data. The imbalanced problems with deep learning models can be solved in a variety of ways. Under-sampling and over-sampling were utilized. One of the primary issues while

Table 8 Feature set from EdgeIIoT dataset

Features
arp.opcode_F1, http.request.method_F2, http.referer_F3, icmp.checksum_F4, icmp.seq_le_F5, icmp.transmit.timestamp_F6, icmp.unused_F7, tcp.ack_F8, tcp.ack_raw_F9, tcp.checksum_F10, tcp.connection.fin_F11, tcp.connection.rst_F12, tcp.connection.syn_F13, tcp.connection.synack_F14, tcp.dstport_F15, tcp.flags.ack,_F16, http.request.version_F17, tcp.len_F18, arp.hw.size_F19, tcp.payload_F20, tcp.seq_F21, http.response_F22, http.tls_port_F23, tcp.flags_F24, udp.stream_F25, dns.qry.name_F26, dns.qry.name.len_F27, dns.qry.qu_F28, dns.qry.type_F29, mqtt.conack.flags_F30, udp.time_delta_F31, mbtcp.trans_id_F32, mbtcp.len_F33, mbtcp.unit_id_F34, mqtt.topic_len_F35, mqtt.msgtype_F36, mqtt.len_F37, mqtt.msg_decoded_as_F38, mqtt.proto_len_F39, mqtt.hdrflags_F40, mqtt.conflag.cleansess_F41, mqtt.conflags_F42, http.content_length_F43, dns.retransmit_request_in_F44, dns.retransmission_F45, dns.retransmit_request_F46, mqtt.topic_F47, mqtt.protoname_F48

Table 9 Five percentage class distribution of EdgeIIoT dataset

Class name	Sample
Normal-C1	560,100
DDoS-C2	413,257
Injection-C3	5493
Malware-C4	64,768
Scanning-C5	8471
MITM-C6	1214
Total No. of observations	1,053,303
Total No. of features	48
Total No. of attack samples	493,203
Total No. of benign samples	560,100

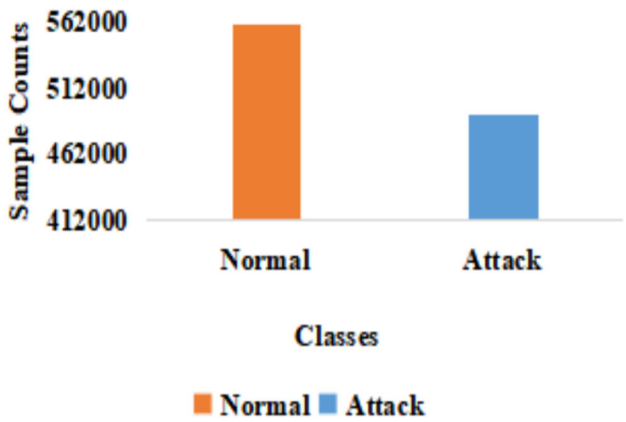


Fig. 2 Data sample distribution statistics of EdgeIIoT dataset

dealing with under-sampling is that it can destroy the important aspect of the data, whereas oversampling can increase the sampling size unnecessarily and cause data distortion. Network traffic data shows a typical data imbalance problem during suspicious activity classification, with a small amount of anomalous traffic and a large amount of regular traffic. Minority classes typically have relatively poor prediction accuracy, but some majority classes show improvements in prediction accuracy when the overall error is reduced.

Table 10 Distribution of samples in the CIC-DDoS2019 dataset

Distribution	Records
Total No. of observations	2,830,743
Total No. of features	84
Total No. of Attack samples	471,454
Total No. of benign samples	2,359,289

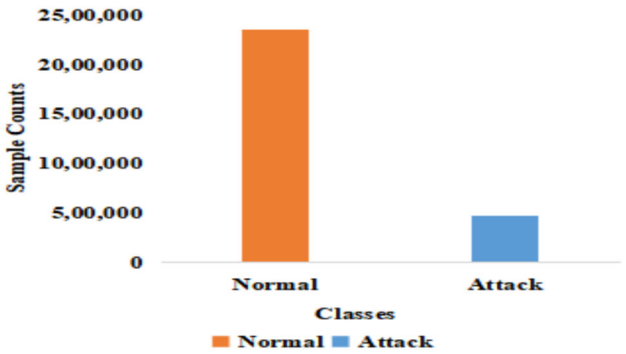


Fig. 3 CIC-DDoS2019 Class imbalances sample distribution

5.2.4.1 Difficult set sampling technique (DSSTE) algorithm Traditional oversampling and under-sampling techniques can introduce noise and limit the effectiveness of training data in IDS, especially when dealing with imbalanced datasets. This is because imbalanced datasets often have a dominant "normal traffic" class that overshadows the minority classes representing actual attacks. As a result, the model struggles to learn the patterns of these minority attack classes, potentially missing critical threats. The Difficult Set Sampling Technique (DSSTE) a hybrid sampling technique, addresses this challenge by strategically manipulating the training data. It reduces the number of samples in the majority class (normal traffic) while eliminating potential noise. This prevents the

Table 11 Feature set from CIC-DDoS2019 dataset

Features
Flow ID-F1, Flow Byts/s-F2, Src IP-F3, Flow Duration-F4, Dst IP-F5, Flow Packets/s-F6, Fwd Seg Size Min-F7, Protocol-F8, Init Bwd Win Byts-F9, ACK Flag Count-F10, Init Fwd Win Byts-F11, SYN Flag Count-F12, Timestamp-F13, CWE Flag Count-F14, Fwd Seg Size Avg-F15, Average Packet Size-F16, Fwd Pkt Len Mean-F17, down/up_ratio-F18, Tot Fwd Pkts-F19, Tot Bwd Packets -F20, Packet Length Variance-F21, Packet Length Std-F22, Bwd Packet Length Max-F23, Bwd Packet Length Mean-F24, Avg Bwd Segment Size-F25, Fwd Packet Length std-F26, Avg Packet Size-F27, bwd_packet_length_std-F28, TotLen Bwd Packets-F29, Subflow Bwd Bytes-F30, Bwd Packets/s-F31, Flow IAT Mean-F32, Idle Max-F33, Dst Port-F34, TotLen Fwd Pkts-F35, Idle Mean-F36, max_packet_length-F37, packet_length_mean-F38, fwd_packet_length_max-F39, fwd_packet_length_min-F40, bwd_packet_length_min-F41, Flow-f42, flow_iat_std-F43, flow_iat_max-F44, flow_iat_min-F45, fwd_iat_total-F46, fwd_iat_mean-F47, fwd_iat_std-F48, fwd_iat_max-F49, fwd_iat_min-F50, bwd_iat_total-F51, bwd_iat_mean-F52, bwd_iat_std-F53, bwd_iat_max-F54, bwd_iat_min-F55, fwd_header_length.1-F56, subflow_fwd_packets-F57, subflow_fwd_bytes-F58, act_data_pkt_fwd-F59, fwd_header_length-F60, bwd_header_length-F61, fwd_packets/s-F62, bwd_packets/s-F63, min_packet_length-F64, active_mean-F65, active_std-F66, active_max-F67, active_min-F68

majority class from overwhelming the minority attack classes. Unlike the majority class, the minority class samples representing attacks remain unchanged for their discrete features. However, for continuous features, DSSTE strategically zooms in or out to amplify the variations within the minority class. This helps the model better recognize the patterns associated with different attack types. By applying these techniques, DSSTE aims to create a more balanced training dataset that accurately reflects the real-world distribution of network traffic. This, in turn, allows the IDS model to learn effectively from both normal and attack patterns, ultimately improving its ability to detect even rare or subtle attacks. Algorithm 1 in Table 12 specifies the pseudocode of DSSTE algorithm.

After dividing the dataset into easy and complex sets. DSSTE algorithm employs various strategies to handle each subset efficiently. To produce a more balanced training set for better classification performance, strategies like removing misclassified examples from the easy set, compressing the majority class in the complex set, and enhancing the minority class are used. Based on the features of the dataset and the demands of the classification task, changes can be made to parameters like the number of nearest neighbor's (K) and the particular compression and augmentation techniques. Figures 4 and 5 shows the balanced class sample distribution of EdgeIIoT and CIC-DDoS2019 datsets.

5.3 Feature selection

High-dimensional feature sets may contain extraneous features that complicate the learning process and add extra noise. A high number of dimensions can also be present in high-dimensional attribute sets. Appropriate feature selection can significantly improve a model's performance and reduces the training time by reducing feature dimensionality. Furthermore, the implementation of feature selection strategies has the potential to mitigate overfitting, which ultimately enhances model generalization [42].

The proposed NIDS framework employs a two-stage feature selection process to identify the most informative features for intrusion detection. This reduces model complexity and improves overall system performance. Random Forest (RF) and XGBoost are powerful machine learning algorithms that can be effectively used for feature selection in various applications, including intrusion detection systems. Both RF and XGBoost provide built-in mechanisms to calculate feature importance scores. RF inherently calculates feature importance scores, indicating each feature's contribution to the model's classification accuracy. Features with higher scores are considered more influential for identifying network intrusions. The AUC-ROC curve is used to evaluate the performance of RF in classifying between attack and normal traffic. By calculating the AUC-ROC score for each feature, we can identify features that effectively distinguish between different classes (attack vs. normal). Features with significant AUC-ROC values (e.g., AUC-ROC > 0) are initially selected as potential candidates for inclusion in the final feature set. Tables 13 and 14 shows the RF algorithm computed AUC value for the 36 features of both the datasets.

After feature shuffling, XGBoost analysis was conducted to assess the importance of chosen features based on their relevance. XGBoost can uncover complex interactions between features, aiding in the identification of relevant feature combinations and capable of assessing the importance of individual features. XGBoost can identify and eliminate redundant or less informative features, leading to a more compact and efficient feature set. This helps to eliminate less informative features and create a more compact and efficient feature set. Based on the XGBoost importance scores, the remaining features are ranked. The features were then ranked based on their relevance, and the top 20 were selected out of 36 features

Table 12 Pseudocode for DSSTE algorithm

Algorithm 1 Pseudocode of DSSTE algorithm

1. Input:
   - Dataset (G) (imbalanced dataset for training)
   - Scaling factor (P) (optional)
2. Output:
   - New training set (SN)
3. Differentiate the complex and easy set:
   - Divide the dataset into two sets: one with comparatively easy to categorize and the other which is more difficult
4. For each instance in the easy set (SE):
   - Evaluate its (K) nearest neighbors
   - Remove instances from (SE) where the majority of their (K)-nearest neighbors belong to a different category
5. Partition into easy set (SE) and complex set (SD):
   -The instances that remain after the removal process are categorized as part of the complex set (SD), and the remaining instances form the easy set (SE)
6. Compress the majority class samples in the complex set:
   - Identify majority class samples in the complex set (SD)
   - Reduce their dominance and balance the class distribution by compressing these majority class samples using an appropriate technique
7. Employ coordinates of compressed majority class samples:
   - To improve learning from the complex set, use the compressed majority class samples' coordinates or representations

Zoom augmentation:

   - For each minority class sample in the complex set (SD):
   - Perform augmentation techniques to generate additional synthetic samples

9. Construct new training set (SN):
   - Assemble the enhanced samples from the complex set (SD) and the modified samples from the easy set (SE) to create the new balanced training set (SN)
10. Output: Return the new training set (SN) for use in classification algorithms

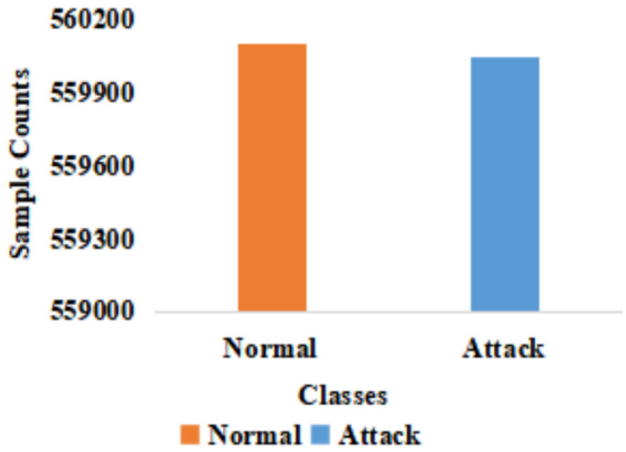


Fig. 4 Statistics of different attack sample distributions of the EdgeIIoT dataset after balancing

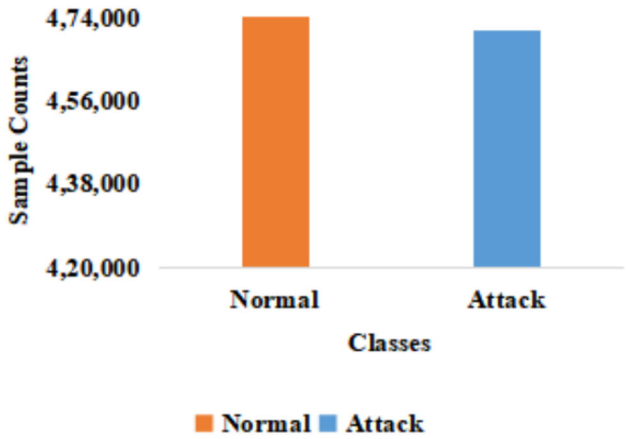


Fig. 5 CIC-DDoS2019 Sample distribution after balancing

from EdgeIIoT and CIC-DDoS2019 dataset. Figures 6 and 7 illustrates the impact of top 20 features selected and the extent of relevance with percentage.

In the second phase, the strengths of filter, wrapper, and embedded methods are leveraged to reduce the initial feature set. Some of the feature selection techniques implemented to reduce the feature set is discussed in the below section:

Table 13 Feature numbers and AUC values EdgeIIoT dataset

Feature number	AUC value	Feature number	AUC value
1	2.297194e−07	17	1.010117e−06
3	3.517579e−07	2	4.902379e−07
4	5.094548e−09	18	9.772338e−08
5	4.399868e−08	20	6.522444e−08
6	2.686325e−08	21	1.669634e−07
10	6.692413e−09	8	8.132809e−07
9	1.621004e−09	24	9.077622e−08
13	3.728309e−09	23	9.313567e−08
12	2.315720e−09	25	3.774624e−08
7	8.463957e−07	26	4.902379e−07
11	9.262879e−12	22	8.116599e−07
14	2.019308e−07	28	6.861478e−07
19	1.614057e−08	29	3.720432e−08
15	7.942920e−08	3	3.732522e−07
16	1.484377e−07	31	2.778846e−08
32	4.321047e−07	34	1.617894e−04
35	2.195303e−07	36	9.763124e−01
38	1.042074e−08	37	2.547322e−05

Table 14 Feature numbers and AUC values of CIC-DDoS2019 dataset

Feature number	AUC value	Feature number	AUC value
4	0.8731	17	0.0459
11	0.5674	26	0.0419
7	0.3592	28	0.0375
29	0.2565	18	0.0341
22	0.1502	16	0.0251
23	0.1849	32	0.0093
24	0.0433	12	0.0071
25	0.0333	33	0.0036
27	0.0132	2	0.0031
34	0.0623	9	0.0023
5	0.0581	15	0.0015
6	0.0183	30	0.0005
35	0.0692	31	0.0003
20	0.0092	1	0.0001
14	0.0101	8	0.0001
13	0.3227	10	0.0001
3	0.2302	21	0.0001
19	0.0875	36	0.0001

5.3.1 Filter based mutual information

Features are ranked according to how well they correlate with the dependent variable in filter-based methods. Because filter techniques don't require model training, they are significantly faster than wrapper methods. Mutual information is the filter method employed in this study. Mutual Information (MI) quantifies the amount of information one random variable (N) reveals about another (O). Mathematically, MI between N and O is defined as:

$$\mathrm{M}\ (\mathrm{N{:}O}) = \mathrm{Z(N)} - \mathrm{Z(N|O)} \tag{2}$$

where M(N:O) is the Mutual Information, Z(N) represents the entropy of variable N, and Z(N|O) represents the conditional entropy of N given O, which reflects the uncertainty remaining in N after knowing the value of O.

5.3.2 Wrapper based RFE

In wrapper methods, a specific machine learning algorithm is used to analyse the provided dataset, thereby determining the feature selection process. By comparing every possible feature combination for evaluation, it uses an avaricious search strategy [43]. Random forest based Recursive Feature Elimination (RFE) wrapper method employed in this investigation. Random Forest's built-in feature importance and handling of noise and irrelevant features make it a good choice for RFE. While not as interpretable as a single decision tree, the features identified by RFE with Random Forest can still provide insights into which features are most important for your model's predictions. Based on the average feature importance scores from the Random Forest, RFE ranks all the features from most important (highest score) to least important (lowest score). It removes the feature with the lowest importance score based on the Random Forest ranking. The remaining features are used to train the model. This process of removing the least important feature and retraining the Random Forest model continues for a predefined number of iterations or until the specified number of features obtained. Although wrapper methods have certain drawbacks, they are nevertheless frequently employed because they can shed light on the dependencies between features and target variables and work well in situations where simple statistical methods struggle to capture the complexity of the relationship. The Algorithm 2 in Table 15 below illustrates the pseudo-code for recursive feature elimination.

5.3.3 TBFI: embedded

Tree-Based Feature Importance (TBFI) is commonly acquired through tree-based ensemble techniques like Gradient Boosting Machines or Random Forest. Unlike certain linear models, these algorithms do not offer a simple mathematical formula for feature importance. In

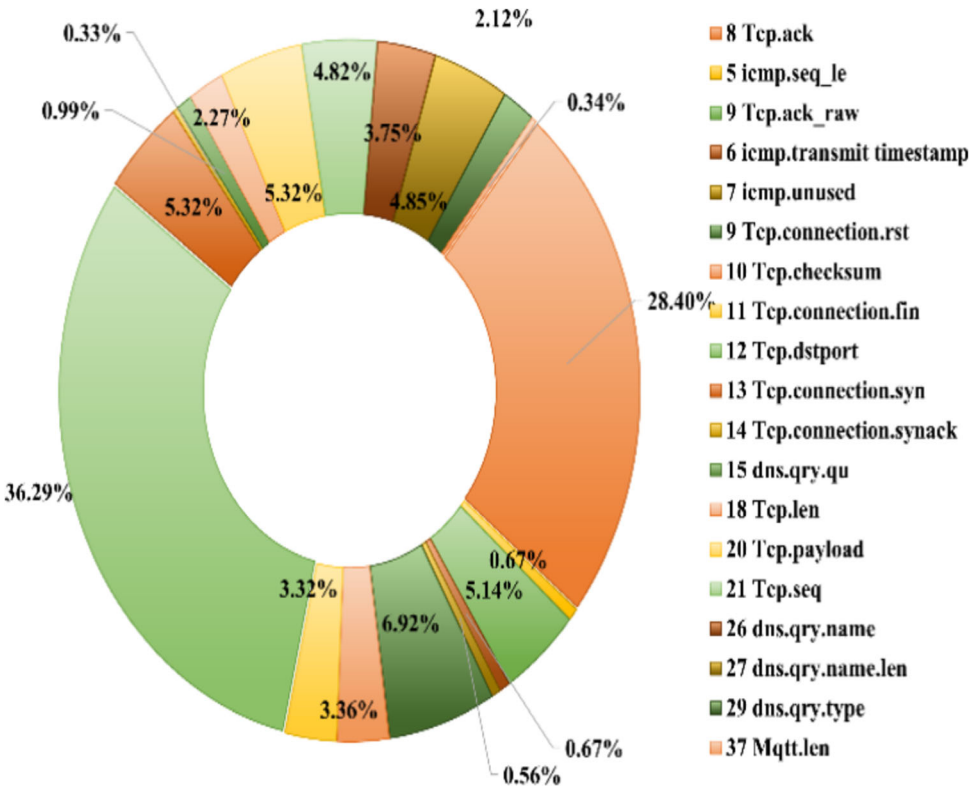


Fig. 6 Ranking of the selected features with high importance from the EdgeIIoT dataset

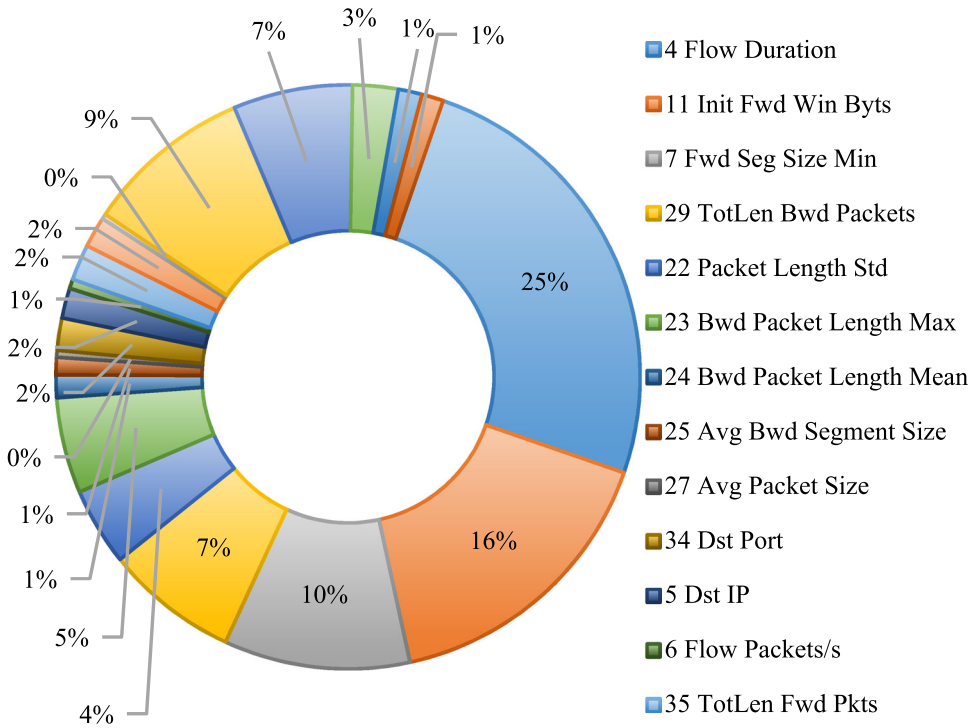


Fig. 7 Ranking of the selected features with high importance from the CIC-DDoS2019 dataset

contrast, the importance of a feature in tree-based models is determined by how each feature helps reduce impurity or increase information gain when decision trees are being constructed within the ensemble. The pseudo-code for Tree Based Feature Importance is illustrated below in Table 16.

Table 15 Pseudocode for RFE algorithm

Algorithm 2 Pseudo code for Recursive Feature Elimination

function RFE(X, y, RF, n_features_to_select):
1. Chosen_features = [] /Initialize a list to keep track of Chosen features:
2. Resting_features = X.columns.tolist() / Set Resting features to all features
3. While (length(Chosen_features < n_features_to_select)
4. model.fit(X[remaining_features], y) / Train a model using the Resting_features
5. Get feature importance or coefficients from the trained model
6. Identify the least important feature among the Resting_features
7. Remove the least important feature from the Resting_features
8. Include the removed feature to Chosen_features
9. Return Chosen_features

5.3.4 Least absolute shrinkage and selection operator (LASSO): filter embedded

Lasso (Least Absolute Shrinkage and Selection Operator) is a regularization technique used for feature selection and regression. It penalizes the absolute size of the coefficients in a linear model, effectively shrinking some coefficients to zero. This leads to sparse models where only a subset of features is retained. The Algorithm 4 in Table 17 illustrates the pseudo-code of LASSO feature selection algorithm.

The regularization parameter, denoted as, regulates the degree of regularization. A greater number of coefficients are set to zero for higher values of α. Address the Optimization Issue: To determine the ideal values of β, minimize the objective function. Feature Selection: In the optimized β vector, features with non-zero coefficients are kept, while those with zero coefficients are eliminated.

The 20 features from phase 1 are further evaluated using the above diverse feature selection techniques from different categories. Tables 18 and 19 showcase the top 12 features from both datasets, which scored 3 and 4. These features were selected using different techniques, with "1" indicating a feature chosen by a particular method and "0" indicating it was not selected. The selected features are highlighted in bold. Finally, a "candidate dataset" was created using only the features that scored at least 3. The threshold can be adjusted based on the specific needs of the analysis and the number of features available. A higher threshold would result in a smaller, more focused dataset containing only the most highly-ranked features. Deep learning classifier models were trained using the candidate dataset to assess their performance. This resulted in a dataset with 12 key features, chosen from the 20 features identified in the second phase (Figs. 8 and 9). Hence, the

Table 16 Pseudocode for TreeBasedFeatureImportance algorithm

Algorithm 3 Pseudocode for TreeBasedFeatureImportance feature selection

function TreeBasedFeatureImportance (X, y):
1.Rf_classifier = RandomForestClassifier()
// Initialize a Random Forest classifier:
2.Rf_classifier.fit(X, y)
// Train the Random Forest classifier on the entire dataset
3.Feature_importances = Rf_classifier.feature_importances
// Get trained classifier feature importances
4. feature_names = X.columns.tolist()
5. feature_importance_dict = {}
6. For each feature and its importance score
7. feature_importance_dict[feature_names[i]] = feature_importances[i]
// Add the feature name and importance score to the dictionary
8. sorted_feature_importance = sort_dict_by_value (feature_importance_dict, reverse = True)
9. Return the sorted_feature_importance list
function sort_dict_by_value(dictionary, reverse):
9. Sort the dictionary by values in descending order if reverse is true, otherwise in ascending order
10. Return the sorted dictionary

Table 17 Pseudocode for Lasso feature selection algorithm

Algorithm 4 Pseudo-code for Lasso feature selection:

function LassoFeatureSelection(X, y, α):

1. Initialize Lasso regression model with regularization parameter α
2. Fit the Lasso model on the dataset (X, y)
3. Obtain the coefficients from the fitted model
4. Identify features with non-zero coefficients
5. Return the selected features

Table 18 Feature selection results of EdgeIIoT dataset

Feature No	Feature name	MI	RFE	TBFI	Lasso	Feature Score
11	**Tcp.connection.fin**	**1**	**1**	**0**	**1**	**3**
8	**Tcp.ack**	**0**	**1**	**1**	**1**	**3**
15	**Tcp.dstport**	**1**	**1**	**1**	**1**	**4**
5	icmp.seq_le	1	0	0	0	1
13	**Tcp.connection.syn**	**1**	**1**	**1**	**0**	**3**
14	Tcp.connection.synack	0	1	0	0	1
26	**dns.qry.name**	**1**	**1**	**0**	**1**	**3**
6	icmp.transmit.timestamp	0	0	0	0	0
12	**Tcp.connection.rst**	**1**	**1**	**0**	**1**	**3**
18	**Tcp.len**	**1**	**1**	**1**	**0**	**3**
9	**Tcp.ack_raw**	**1**	**1**	**1**	**0**	**3**
10	**Tcp.checksum**	**1**	**1**	**1**	**1**	**4**
37	Mqtt.len	1	1	0	0	2
28	**dns.qry.qu**	**1**	**0**	**1**	**0**	**3**
27	**dns.qry.name.len**	**1**	**1**	**0**	**1**	**3**
4	Icmp.checksum	1	0	0	0	1
20	**Tcp.payload**	**1**	**1**	**1**	**1**	**4**
21	Tcp.seg	0	1	0	1	2
7	icmp.unused	0	0	0	1	1
29	dns.qry.type	1	0	0	1	2

resultant 12 number of features were used to train deep learning models.

5.4 Experimental setup

Our experiment utilized an MSI GF63 10SCXR laptop equipped with Windows 11. The core of the system was an Intel(R) Core(TM) i5-10300U CPU, providing a base clock speed of 2.5 GHz and the ability to boost up to even higher frequencies. To develop, train, and assess the deep learning model, we leveraged a suite of open-source libraries, including Pandas, NumPy, PyTorch, Seaborn, and Catalyst. The data was meticulously divided for training, validation, and testing purposes, with a 70:30 split employed. To ensure randomness, we used ten distinct random state values during the splitting process. This meticulously prepared dataset served as the foundation for training the deep learning models, and we subsequently evaluated their performance using diverse metrics.

5.4.1 Performance metrics

To comprehensively evaluate the performance of the proposed IIoT-NIDS model, we employed a diverse set of metrics. These metrics included Matthew's Correlation Coefficient (MCC), Brier Score, accuracy, precision, recall, F1 score, and false alarm rate. The mathematical formulas for these metrics can be found in Eqs. (3) to (9). To compare the model's performance, we evaluated it on two datasets: one containing all features and another containing a reduced set of candidate features. The metrics for each dataset were calculated using the following parameters:

- True Positives (TPs): Intrusion events correctly identified by the model.
- True Negatives (TNs): Normal data points correctly classified as normal traffic.
- False Positives (FPs): Normal data points incorrectly flagged as intrusions.

Table 19 Feature selection results of CIC-DDoS2019 dataset

Feature No	Feature name	MI	RFE	TBFI	Lasso	Feature Score
4	**Flow Duration**	**1**	**1**	**1**	**0**	**3**
11	**Init Fwd Win Byts**	**1**	**1**	**1**	**1**	**4**
7	**Fwd Seg Size Min**	**1**	**0**	**1**	**1**	**3**
29	TotLen Bwd Packets	1	0	0	1	2
22	Packet Length Std	**1**	**0**	**0**	**0**	**1**
23	Bwd Packet Length Max	1	0	1	0	2
24	Bwd Packet Length Mean	**0**	**0**	**1**	**1**	**2**
25	**Avg Bwd Segment Size**	1	1	1	1	4
27	**Avg Packet Size**	**1**	**1**	**1**	**1**	**4**
34	**Dst Port**	**1**	**0**	**1**	**1**	**3**
5	**Dst IP**	**1**	**1**	**0**	**1**	**3**
6	**Flow Packets/s**	**1**	**1**	**1**	**1**	**4**
35	**TotLen Fwd Pkts**	0	1	1	1	3
20	Tot Bwd Packets	**0**	**0**	**1**	**0**	**1**
14	CWE Flag Count	**0**	**1**	**0**	**1**	**2**
13	**Timestamp**	1	1	1	1	4
3	**Src IP**	**1**	**0**	**1**	**1**	**3**
19	**Tot Fwd Pkts**	0	1	1	1	3
17	Fwd Pkt Len Mean	0	0	1	0	1
26	Fwd Packet Length std	0	0	1	0	1

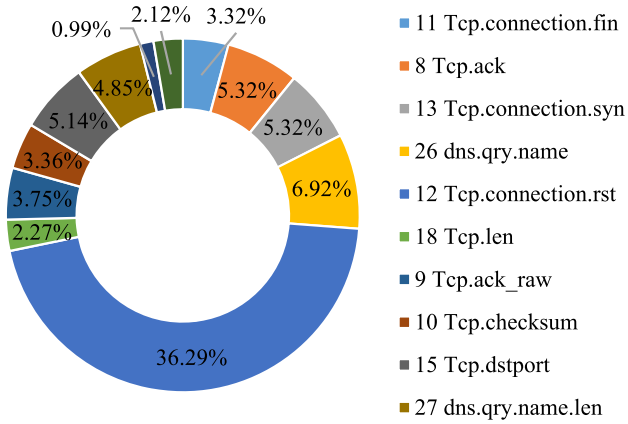


Fig. 8 12 features selected with high importance from EdgeIIoT dataset

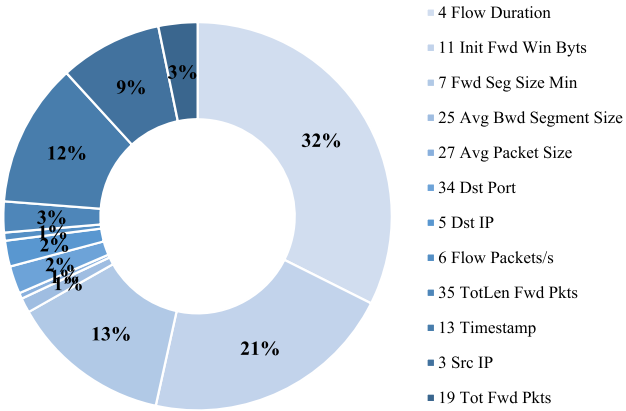


Fig. 9 12 features selected with high importance from CIC-DDoS2019 dataset

- False Negatives (FNs): Intrusion events that the model missed (undetected attacks).

Evaluating these metrics on both datasets provides valuable insights. This allows us to assess the effectiveness of the feature selection process in improving the overall performance of the IIoT-NIDS model.

$$\text{Accuracy,}\quad CC = \frac{(TP + TN)}{(TP + TN + FP + FN)} \tag{3}$$

$$\text{Recall,}\quad Rec = \frac{(TP)}{(TP + FN)} \tag{4}$$

$$\text{Precision,}\quad Prec = \frac{(TP)}{(TP + FP)} \tag{5}$$

$$\text{F1-Score,}\quad F1 = \frac{(2 * Recall * Precision)}{(Recall + Precision)} \tag{6}$$

$$\text{False Alarm Rate,}\quad FAR = \frac{(FP)}{(TN + FP)} \tag{7}$$

While prior research has often emphasized accuracy alone, it's crucial to consider additional performance measures for intrusion detection systems (IDS) due to the inherent class imbalance problem. In IDS, normal traffic significantly outweighs attack traffic. The proposed work addresses this

challenge by incorporating a wider range of metrics beyond just accuracy. Including both MCC and Brier Score alongside other metrics provides a more holistic understanding of the proposed IDS model's performance. MCC is particularly effective for imbalanced datasets, a common characteristic of IDS data. Unlike accuracy, which can be misleading in such cases, MCC factors in true positives, negatives, and both types of errors (false positives and negatives). This results in a more balanced assessment of the model's performance. The mathematical formula for MCC is as follows:

$$\mathrm{MCC} = \frac{(\mathrm{TP} * \mathrm{TN} - \mathrm{FP} * \mathrm{FN})}{\sqrt{(\mathrm{TP} + \mathrm{FP})(\mathrm{TP} + \mathrm{FN})(\mathrm{TN} + \mathrm{FP})(\mathrm{TN} + \mathrm{FN})}} \tag{8}$$

The Brier Score is a metric used to assess the accuracy of probabilistic predictions in classification tasks, particularly valuable for intrusion detection systems (IDS). It calculates the average squared difference between the predicted probabilities (how likely an event is) and the actual outcomes (normal traffic or attack). A lower Brier Score indicates better model performance. Beyond just accuracy, the Brier Score also reflects the model's calibration. Calibration refers to how well the assigned probabilities correspond to the actual likelihood of attacks. In simpler terms, it tells us if the model is overconfident or less confident in its predictions.

Mathematically, the Brier Score for a set of predictions (P) and their corresponding binary outcomes (O), where normal traffic is 0 and attack is 1, is calculated as follows:

$$\text{Brier Score, } \mathrm{BS} = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} (\mathrm{P_i} - \mathrm{O_i})^2 \tag{9}$$

where N is the total number of samples or instances, P_i is the predicted probability for sample i, O_i is the actual binary outcome for sample i, Where $O_i = 1$ for a positive outcome and $O_i = 0$ for a negative outcome.

6 Results and discussions

This study utilizes EdgeIIoT dataset with 20 million records and CIC-DDoS2019 dataset containing 2.83 million records. For analysis, a subset of the preprocessed EdgeIIoT data was chosen, consisting of 1,047,684 records with 36 features and CIC-DDoS2019 dataset containing 2,830,743 records with 36 features encompassing various attack categories. The training process involved dividing the candidate dataset into two portions: 70% for training and 30% for testing the classifier models performance. The classifiers discussed in Sect. 4.1 were employed during both the training and testing phases. For evaluating diverse learning models 733,379 training and 314,305 testing samples of EdgeIIoT dataset, and 1,981,520 training and 849,222 testing samples of CIC-DDoS2019 were used. A candidate dataset with most relevant features for training deep learning (DL) models was created. This candidate dataset includes all 36 features and also contains features with a feature score at least 3, resulting in a subset of 12 attributes. To comprehensively assess the performance of the models using features from the candidate dataset, various metrics were utilized. These metrics included MCC, Brier Score, accuracy, precision, recall, F1 score, and false alarm rate.

Here's a breakdown of the performance for each model with 36 features of EdgeIIoT and CIC-DDoS2019 dataset. In case of EdgeIIoT dataset, GNN attained an outstanding accuracy of 99.69%. Remarkably, it also demonstrated high precision and F1-score, indicating its effectiveness in balancing the identification of attacks with the minimization of false positives. DQN showed an accuracy of 99.32%. DNN delivered an accuracy of 99.48%. GAN achieved the highest accuracy among these models (99.69%), its F1-score exhibited some variation. This suggests a potential trade-off between accuracy and precision. For CIC-DDoS2019 dataset, All classifiers demonstrate strong performance, with accuracy scores ranging from 95.37 to 99.89%. GAN consistently achieves the highest accuracy, precision, and recall suggesting its effectiveness in capturing complex data patterns and its ability to accurately identify positive and negative instances. GNN shows competitive results, especially in terms of recall, suggesting its effectiveness in capturing graph-based relationships in the data. GAN and DNN achieving the highest MCC, suggesting a strong correlation between the predicted and actual classes. GAN has the lowest Brier Score, indicating better calibration of predicted probabilities.

Our IDS effectively detects malicious traffic using a substantially smaller set of features, without compromising accuracy. This emphasizes the importance of choosing the right feature selection method for optimal performance. The proposed feature selection method considers only features with feature score at least 3, thereby achieving promising results. Notably, the presented method achieved a superior accuracy of 99.9% compared to other classifiers discussed in Sect. 4.1. False Alarm Rate (FAR) is a crucial metric in evaluating the performance of a classification model, particularly in applications like intrusion detection systems. It directly impacts the effectiveness and efficiency of these systems. False Alarm Rate (FAR) is calculated as the ratio of incorrectly classified negative instances (false positives) to the total number of actual negative instances (true negatives). The GAN model demonstrated exceptional performance in terms of false alarm rate (FAR), achieving a remarkably low 0.04% compared to the LSTM

model's FAR of 0.47% on the EdgeIIoT dataset. Compared to other classifiers, the GAN model demonstrated exceptional performance with a remarkably low False Alarm Rate (FAR) of 0.11%, while LSTM achieved a respectable FAR of 0.71% on the CIC-DDoS2019 dataset. Since, both the datasets is highly imbalanced, relying solely on accuracy can be misleading. The results for both datasets are summarized in Tables 20 and 21, with corresponding visualizations provided in Figs. 10 and 11.

To effectively compare the performance of various DL models using the candidate dataset, this study primarily focuses on two key metrics: Matthews Correlation Coefficient (MCC) and Brier Score. For efficient anomaly detection, classifier models were trained using a candidate dataset containing the top 12 most important features. The candidate dataset itself contains 12 features with feature score at least 3, chosen through the previously described feature selection method. Only MCC (Matthews Correlation Coefficient) and Brier score metrics were considered when comparing the performance of all the classifier models. The comparison of MCC scores for all deep learning (DL) models with key features is presented in Table 22. In a perfect binary classification scenario, the MCC value ranges from + 1 (ideal) to − 1 (worst). As shown in Table 22, the MCC values for all algorithms in this experiment fall within the range of 0, indicating a good performance in correctly classifying both normal and attack traffic for both the datasets.

The high accuracy of GAN, GNN, and DNN models on both the dataset (as shown in Tables 22 and 23) is further supported by their MCC values. The absence of negative MCC values (− 1) suggests a well-balanced classification between normal and attack traffic. However, the MCC value for the LSTM model reveals a potential issue. Its MCC value falls outside the typical range, indicating a higher level of imbalance compared to the other models. This highlights the importance of using MCC alongside accuracy, especially when dealing with imbalanced datasets. Even if a model shows high accuracy, a poor MCC can indicate a bias towards the majority class (normal traffic) and a potential inability to accurately detect attacks. Tables 22 and 23 showcases the MCC scores achieved by all the DL models. Interestingly, the results reveal that the DQN model's MCC score actually improved when using the reduced candidate dataset. It obtained an MCC of 0.993 with all features, and a slightly higher score

Table 20 Results for EdgeIIoT dataset with 36 features (Binary-mode)

Classifiers	LSTM	GAN	GNN	DQN	DNN
Accuracy	93.87	99.71	95.69	99.32	97.48
Precision	94.65	99.54	99.87	99.12	98.43
Recall	91.47	97.99	98.12	99.81	99.45
F1-score	91.91	98.59	99.67	99.57	99.31
FAR	0.65	0.04	0.47	0.26	0.17
MCC	0.919	0.991	0.979	0.995	0.929
BRIER SCORE	0.47	0.01	0.33	0.17	0.10

Table 21 Results for CIC-DDoS2019 dataset with 36 features (Binary-mode)

CLASSIFIERS	LSTM	GAN	GNN	DQN	DNN
ACCURACY	95.37	99.89	97.62	99.12	98.38
PRECISION	95.65	99.78	97.51	99.28	98.11
RECALL	94.47	98.52	97.75	99.41	98.75
F1-SCORE	94.91	99.37	97.37	99.73	98.61
FAR	0.71	0.11	0.47	0.32	0.19
MCC	0.959	0.995	0.979	0.991	0.989
Brier score	0.35	0.15	0.33	0.27	0.24

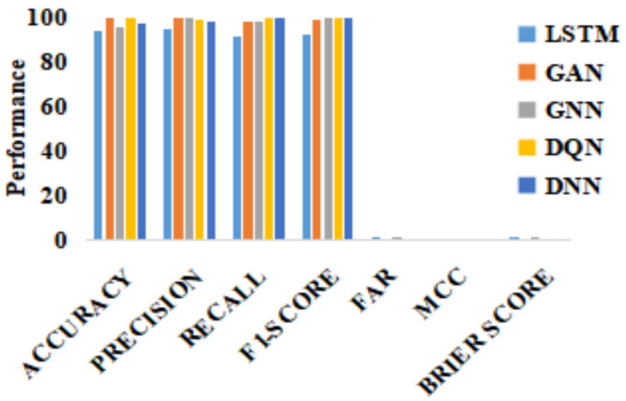


Fig. 10 Comparison of the performances of the proposed models on EdgeIIoT dataset

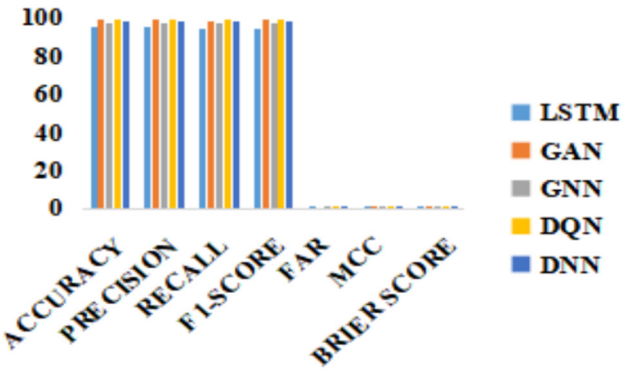


Fig. 11 Comparison of the performances of the proposed models on CIC-DDoS2019 dataset

Table 22 Comparison of the MCC score of EdgeIIoT dataset for all the classifier models

Classifier	36 features	Feature score ≥ 3 (12 features)
LSTM	0.989	0.990
GAN	0.999	0.999
GNN	0.987	0.988
DQN	0.993	0.995
DNN	0.995	0.994

Table 23 Comparison of the MCC score of CIC-DDoS2019 dataset for all the classifier models

Classifier	36 features	Feature score ≥ 3 (12 Features)
LSTM	0.959	0.950
GAN	0.995	0.995
GNN	0.979	0.978
DQN	0.991	0.990
DNN	0.989	0.985

of 0.995 with the candidate set containing features with feature score at least 3. Similarly, the GAN model exhibited strong performance on both datasets. While its accuracy for the full feature set is mentioned as 0.999, the performance on the candidate dataset of features with feature score at least 3 remains impressive. Training DL models with carefully chosen key features resulted in a slight improvement in their detection rate compared to models trained without these features.

GAN consistently demonstrates superior performance with MCC scores of 0.995 indicating excellent agreement between predicted and actual classes on CIC-DDoS2019 dataset. LSTM and GNN experience a slight drop in MCC, indicating that some features might be redundant or less informative. DQN performs well in terms of accuracy and F1-score but has a relatively high false alarm rate (FAR). Reducing the feature set to those with scores ≥ 3 has a minimal effect on MCC for most models, suggesting that the selected features are highly informative. The MCC scores for both datasets are visualized in histogram form in Figs. 12 and 13.

The lower the brier score the more accurate the prediction outcome is. Comparatively GAN model resulted in very less brier score as 0.2% which predicted more accurate outcome. The Brier scores for the GAN model exhibited some variation. However, a key finding is that the candidate dataset of features with feature score at least 3 performed better when compared to the dataset containing all features. These findings, summarized in Tables 24 and

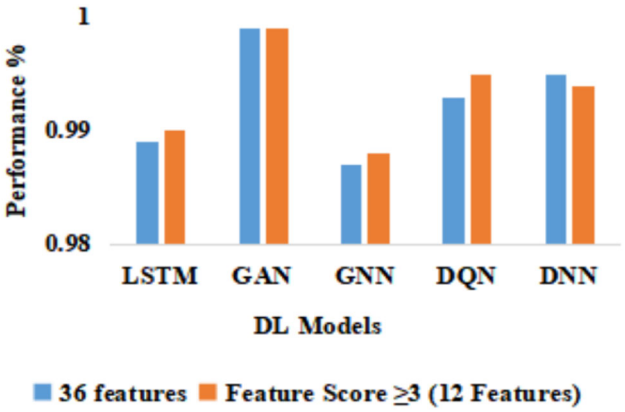


Fig. 12 Comparison of the MCC score of EdgeIIoT dataset for all the proposed classifier models

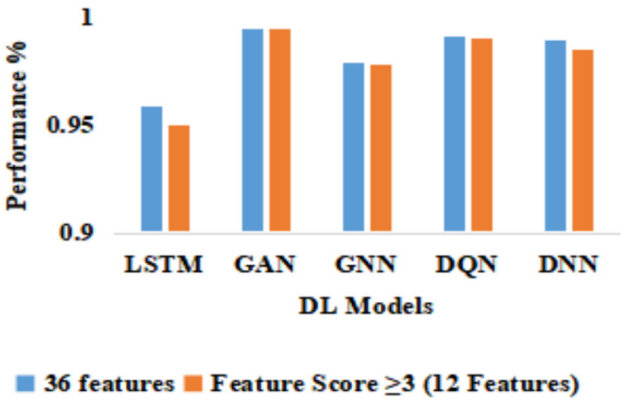


Fig. 13 Comparison of the MCC score of CIC-DDoS2019 dataset for all the proposed classifier models

25, further validate the effectiveness of our proposed method. This is evident from the Brier score results presented in Figs. 14 and 15. By utilizing a reduced set of carefully chosen features, our IDS model achieved a lower Brier score, indicating a more accurate calibration of predicted probabilities for attack detection. This highlights the advantage of our feature selection method in achieving better performance with a smaller feature set.

Table 24 Comparison of the Brier score for all the classifier models on EdgeIIoT dataset

Classifier	36 features	Feature Score ≥ 3 (12 features)
LSTM	0.47	0.46
GAN	0.01	0.03
GNN	0.33	0.3
DQN	0.27	0.26
DNN	0.3	0.2

Table 25 Comparison of the Brier score for all the classifier models on CIC-DDoS2019 dataset

Classifier	36 features	Feature score ≥ 3 (12 features)
LSTM	0.35	0.34
GAN	0.15	0.14
GNN	0.33	0.30
DQN	0.27	0.26
DNN	0.24	0.21

The consistently low Brier scores from GAN on CIC-DDoS2019 dataset highlight its ability to produce well-calibrated probability estimates. The minimal impact of feature reduction suggests that the selected features are informative and contribute to accurate predictions. Reducing features to those with scores ≥ 3 has a limited effect on the Brier score, suggesting that the removed features were less critical for model calibration. DQN's Brier score improves slightly when using fewer features, indicating that some features might introduce noise or redundancy.

Overall, the results demonstrate that the candidate dataset with a reduced number of features performed competitively with the dataset containing all features.

This study demonstrates that the candidate NIDS can effectively identify malicious traffic using a reduced set of features, without compromising the performance aspects. This is achieved by employing a well-designed feature selection method that chooses the most relevant attributes for attack detection. Presenting our model's performance metrics in a Table 26, alongside the results from the identified studies. In Table 26, the work efficacy is compared to [19, 37, 38]. Attack classification framework efficacy was assessed using BoT-IoT, CICIDS2018, and CIDDS-001 datasets. Accuracy was 99.5, 99.49 and 99.83%, but other performance metrics were not studied except in [19] a good MCC of 99.62% was achieved. The proposed models significantly outperform the baselines [30–32] in terms of accuracy, precision, recall, and F1-score. The proposed GAN model achieved an outstanding results on the EdgeIIoT and CIC-DDoS2019 dataset, highlighting their ability to handle diverse intrusion patterns. While all the results are not provided in the study [32] on X-IIoTID dataset, the reported accuracy of 99.84% for CNN + LSTM is impressive. While the results are not directly comparable due to potential dataset differences, the high accuracy achieved by the hybrid model on EdgeIIoT and CIC-DDoS2019 suggests its potential for other datasets.

The framework in [30, 31, 39, 40, 44, 45] uses the same dataset and algorithms as ours, making it somewhat relevant to the study. The Densenet model [45] scored 94.94% accuracy, 98.3% precision, 92.4% recall, and 95.3% f1-score but the most effective MCC and Brier score performances were not included in the study. Which is crucial in intrusion detection. While the CNN + LSTM model [44] achieved an impressive 100% accuracy, it's essential to consider the potential impact of class imbalance in the dataset. Further analysis, including examining precision, recall, and F1-score, is necessary to assess the model's true performance. But, those study was not conducted in the experiment. Given that [37] utilizes the same dataset as our work, it represents the most pertinent approach. Our algorithms delivered superior results, achieving high

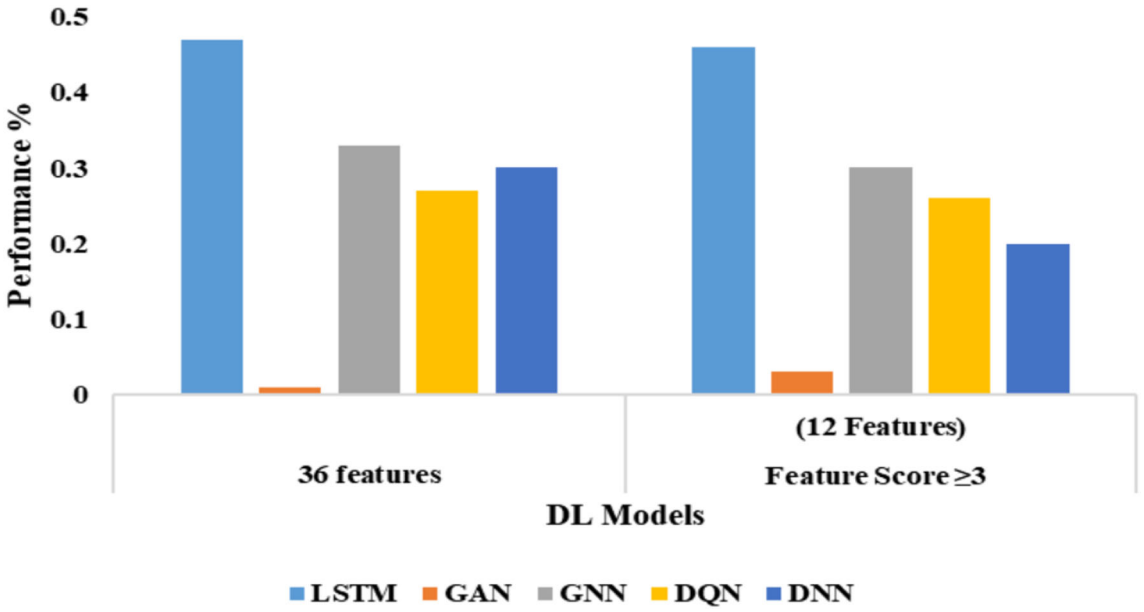


Fig. 14 Comparison of the Brier score on EdgeIIoT dataset for all the proposed classifier models

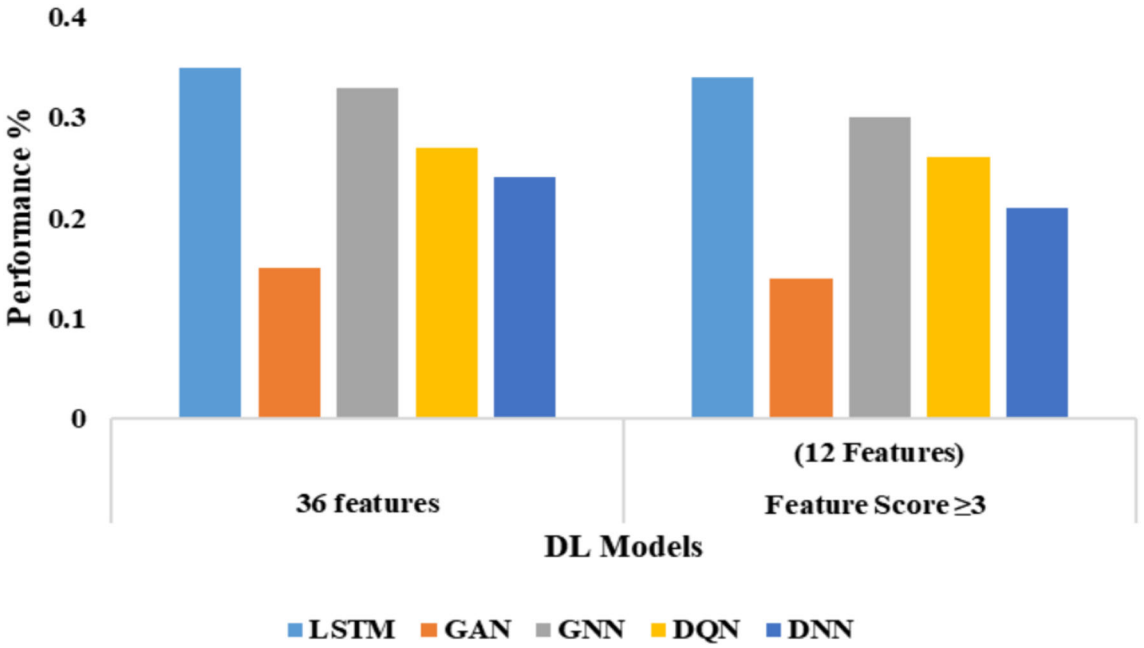


Fig. 15 Comparison of the Brier score of CIC-DDoS2019 dataset for all the proposed classifier models

accuracy and F1-score, which are crucial metrics for intrusion detection. For instance, our GAN-based implementation achieved an accuracy of 99.9%, outperforming the range of 55.13–75.07% achieved by DT and RF in [40], but failed to describe the classification pipeline preprocessing. Other factors that may explain the performance difference include algorithm implementations. Our framework was the most accurate and consistent when using different DL algorithms on EdgeIIoT and CIC-DDoS2019 dataset. More than 90% of literature models were accurate. Our results are consistent across performance metrics. The framework in [39] uses the same dataset and algorithms as ours, making it some-what relevant. Decision trees had the highest classification accuracy of 99.86% and MCC of 99.6%. But our GAN model with feature score greater than 3 out performed with accuracy (99.9%) and MCC (99.9%). But the most reliable classification metric MCC and brier score which we achieved to be (0.2–0.47%) was not a part of their study.

Our models and recent EdgeIIoT and CIC-DDoS2019 dataset using works [39, 40, 44, 45] are compared for the performances in Table 26. Figure 16 displays the results of presented DL models along with the state-of-the-art advanced learning models. Our analysis using the EdgeIIoT and CIC-DDoS2019 dataset, despite the introduction of new attack types the proposed model consistently outperforms existing methods for attack detection, demonstrating its robustness and adaptability to diverse threat landscapes.

While the proposed framework demonstrates promising results, certain limitations and future research directions are worth considering. The performance of the proposed NIDS framework heavily relies on the quality and diversity of the training datasets. A limited or biased dataset may impact the model's generalization capabilities. Though the proposed approach addresses the computational constraints of edge devices, deep learning models can still be computationally intensive. The effectiveness of the NIDS may be compromised by emerging and sophisticated cyberattacks. Continuous monitoring and model retraining are crucial to adapting to evolving threats. Collecting a larger and more diverse dataset will help in enhancing the model's generalizability and robustness. Exploring techniques like model pruning, quantization, optimization, and knowledge distillation to reduce model complexity and improve efficiency. Testing the NIDS on a variety of edge devices with different computational capabilities to assess its adaptability. Investigating adaptive learning techniques to enable the NIDS to adapt to changing network conditions and emerging threats. Developing techniques to interpret the decisions made by the deep learning models to enhance transparency and trust.

7 Conclusion

This paper proposes a robust and lightweight deep learning-enabled Network Intrusion Detection System (NIDS) specifically designed for edge-enabled Industrial Internet of Things (IIoT) applications. The presented framework incorporates various aspects to ensure reliable cyberattack detection on such resource-constrained platforms. First, a

Table 26 Comparison of the performance of the proposed model with the state-of-the art technologies

References	Methodology	Dataset	Accuracy	Precision	Recall	F1-Score	FAR	MCC	Brier Score
[19]	SMOTE-DRNN	BoT-IoT	99.5%	–	–	–	–	99.62%	–
[30]	Hybrid CNN + LSTM	EdgeIIoT	97.85%	98.13%	97.59%	97.81%	–	–	–
[31]	DL Models	EdgeIIoT	98.32%	98.78%	97.22%	–	–	–	–
[32]	CNN + LSTM	X-IIoTID	99.84%	–	–	–	–	–	–
[34]	Blockchain-FedIoT	UNSW-NB15	99%	99%	99%	99%	–	–	–
[35]	DRL	NSL-KDD	94%	–	–	80.84%	–	–	–
[36]	Homomorphic Encryption based FL	UNSW-NB15	99%	99%	99%	99%	–	–	–
[37]	ML Models	CICIDS2018	99.49%	–	–	–	–	–	–
[38]	CNN + LSTM	CIDDS-001	99.83%	–	–	–	–	–	–
[44]	CNN + LSTM	EdgeIIoT	100%	–	–	–	–	–	–
[40]	DT	EdgeIIoT	55.13%	–	–	67.11%	–	–	–
	RF		75.07%	–	–	80.83%	–	–	–
	KNN		74.87%	–	–	79.18%	–	–	–
	SVM		77.33%	–	–	77.61%	–	–	–
	DNN		94.67%	–	–	77.33%	–	–	–
[45]	Densenet	EdgeIIoT	94.94%	98.3%	92.4%	95.3%	–	–	–
[39]	ROC based Hybrid model	EdgeIIoT	99.86%	–	–	–	–	99.6%	NA
		CIC-DDoS2019	99.16%	–	–	–	–	96.8%	NA
Proposed work	LSTM	EdgeIIoT	98.91%	98.65%	98.47%	98.87%	0.65	0.99	0.46
	GAN		99.9%	99.54%	99.99%	98.59%	0.04	0.999	0.03
	GNN		99.69%	99.87%	99.12%	99.67%	0.47	0.988	0.3
	DQN		99.32%	99.12%	99.81%	99.57%	0.26	0.995	0.26
	DNN		99.48%	98.43%	99.45%	99.31%	0.17	0.994	0.2
Proposed Work	LSTM	CIC-DDoS2019	95.37%	95.65%	94.47%	94.91%	0.71	0.959	0.34
	GAN		99.89%	99.78%	98.52%	99.37%	0.11	0.995	0.14
	GNN		97.62%	97.51%	97.75%	97.37%	0.47	0.979	0.30
	DQN		99.12%	99.28%	99.41%	99.73%	0.32	0.991	0.26
	DNN		98.38%	98.11%	98.75%	98.61%	0.19	0.989	0.21

technique called DSSTE sampling, well-suited for anomaly detection, is used. Then, a permutation-based feature importance method with the AUC metric is applied on an RF model to identify the most relevant features. Finally, XGBoost's built-in feature importance scoring further refines the feature set. This multi-pronged approach leverages the strengths of various selection methods, resulting in a curated set of 12 key features. These 12 features are then used to train different deep learning anomaly detection models. To evaluate their effectiveness, the results are presented using confusion matrix metrics, along with the Matthews Correlation Coefficient (MCC) and Brier score. Since both MCC and Brier score are particularly suitable for binary classification tasks, they offer a more accurate assessment of the models' performance in detecting cyberattacks. Among the evaluated models, the Generative Adversarial Network (GAN) demonstrated the most promising results. To validate these findings, the system's performance is compared with state-of-the-art works in the field. The comprehensive comparison and detailed methodology effectively demonstrate the proposed NIDS framework's suitability for real-time cyberattack detection in edge-enabled IIoT environments. Future research will focus on integrating explainable AI techniques to enhance the transparency and interpretability of the intrusion detection system. This will enable users to better understand the model's decision-making process and build trust. Additionally, exploring federated learning can help preserve data privacy by training the model on distributed edge devices without sharing sensitive data. To address emerging threats, continuous monitoring of evolving attack landscapes and corresponding model updates are essential. By incorporating these

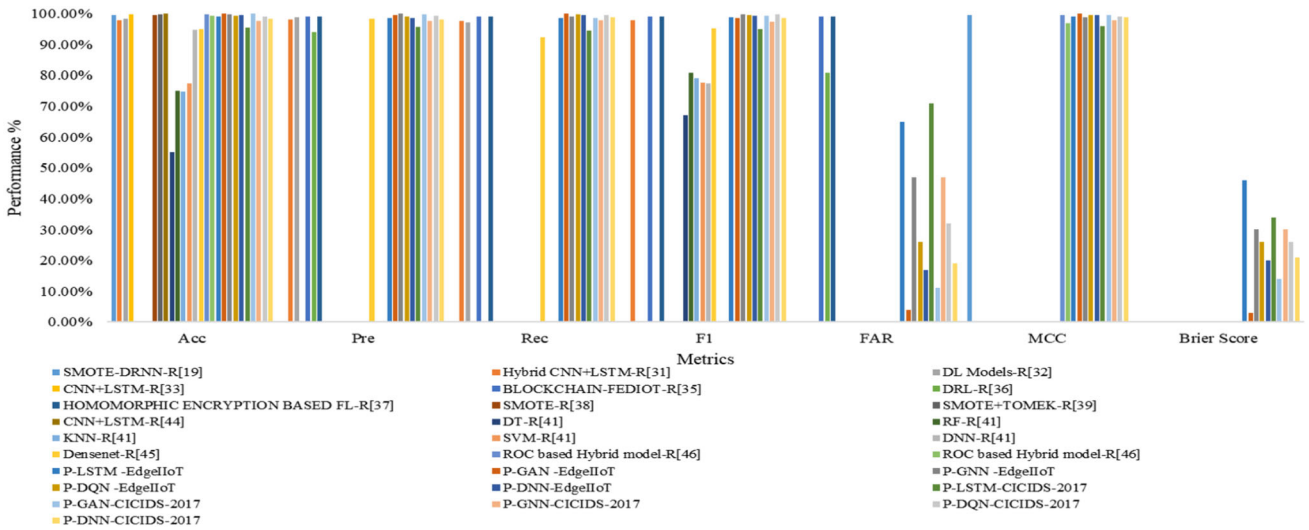


Fig. 16 A comparative analysis of the proposed models against existing models

advancements, the proposed NIDS can further strengthen security and privacy in edge-based environments.

Acknowledgements We would like to express our sincere gratitude to the Vellore Institute of Technology for their invaluable support and assistance in the completion of this research work.

Author Contributions Mangayarkarasi Ramaiah and Mohemmed Yousuf Rahamathulla did conceptualization and methodology; Mohemmed Yousuf Rahamathulla performed formal analysis and investigation; drafting, review, and editing were done by Mohemmed Yousuf Rahamathulla and Mangayarkarasi Ramaiah.

Funding The author confirms that there is no ties to any group that might benefit monetarily or otherwise from the publication of this manuscript, either directly or indirectly.

Data availability The datasets used and/or created for this study can be found at [https://ieee-dataport.org/documents/edge-iiotset-new-comprehensive-realistic-cyber-security-dataset-iot-and-iiot-applications#main-content], a repository with the [Edge-IIoTset dataset.zip], [https://www.unb.ca/cic/datasets/ddos-2019.html].

Declarations

Conflict of interest The authors state that they have no financial or personal stake in the results of this study being made public.

Ethical approval Validation none of the authors of this article conducted any experiments involving human or animal subjects.

References

1. Simone, R., Maurantonio, C., Roberto, P.: Intrusion detection at the network edge: solutions, limitations, and future directions. In: International Conference on Edge Computing, pp. 59–75 (2019)
2. Furnell, S.: Enemies within: the problem of insider attacks. Comput. Fraud Secur. 6–11 (2004)
3. Vermesan, O.: Internet of Things Strategic Research and Innovation Agenda, pp. 7–151. River Publishers, New York (2022)
4. Harbi, Y., Aliouat, Z., Refoufi, A., Harous, S.: Recent security trends in internet of things: a comprehensive survey. IEEE Access **9**, 113292–113314 (2021)
5. Bangui, H., Buhnova, B.: Lightweight intrusion detection for edge computing networks using deep forest and bio-inspired algorithms. Comput. Electr. Eng. **100**, 107901 (2022)
6. Harbi, Y., Aliouat, Z., Harous, S.: Fog computing security and privacy for internet of things (IoT) and industrial internet of things (IIoT) applications: state of the art. In: Security Issues in Fog Computing from 5G to 6G: Architectures, Applications and Solutions, pp. 145–157 (2022)
7. Gopi, R., Mathapati, M., Prasad, B., Ahmad, S., AlWesabi, F.N., Alohali, M.A., Hilal, A.M.: Intelligent dos attack detection with congestion control technique for vanets. Comput. Mater. Continua **72**(1), 141–156 (2022)
8. Fedorchenko, E., Novikova, E., Shulepov, A.: Comparative review of the intrusion detection systems based on federated learning: advantages and open challenges. Algorithms **15**(7), 247 (2022)
9. Hindy, H., Brosset, D., Baynea, E., Seeam, A.K., Tachtatzis, C., Atkinson, R., Bellekens, X.: A taxonomy of network threats and the effect of current datasets on intrusion detection systems. IEEE Access **8**, 104650–104675 (2020)
10. Ramaiah, M., Chandrasekaran, V.: An intrusion detection system using optimized deep neural network architecture. Trans. Emerg. Telecommun. Technol. **32**(4), e4221 (2021)
11. Ramaiah, M., Vanmathi, C.: An efficient intrusion detection system to combat cyber threats using a deep neural network model. J. ICT Res. Appl. **17**(3) (2023)
12. Mangayarkarasi, R., Vanmathi, C., Ravi, V.: A robust malware traffic classifier to combat security breaches in industry 4.0 applications. Concurr. Comput. Pract. Exp. **35**(23), e7772 (2023)
13. Pektaş, A.A., Acarman, T.: Deep learning to detect botnet via network flow summaries. Neural Comput. Appl. **31**, 8021–8033 (2018)
14. Ahmad, S., Jha, S., Alam, A., Alharbi, M., Nazeer, J.: Analysis of intrusion detection approaches for network traffic anomalies with comparative analysis on botnets. Secur. Commun. Netw **2022**, 1–11 (2022)
15. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications: centralized and federated learning | IEEE DataPort (ieee-dataport.org)

16. Fan, S., Sohail, S.F., Gu, X.: A survey on the edge computing for the internet of things sampling-based machine learning models for intrusion detection in imbalanced dataset. Electronics **13**(10), 1878 (2024)
17. Bagui, S., Li, K.: Resampling imbalanced data for network intrusion detection datasets. J. Big Data **8**(1), 6 (2021)
18. Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A.: Developing realistic distributed denial of service (ddos) attack dataset and taxonomy. In: IEEE 53rd International Carnahan Conference on Security Technology (2019)
19. Popoola, S.I., Adebisi, B., Ande, R., Hammoudeh, M., Anoh, K., Atayero, A.A.: Smote-drnn: A deep learning algorithm for botnet detection in the internet-of-things networks. Sensors **21**, 1–9 (2021)
20. Wu, T., Fan, H., Zhu, H., You, C., Zhou, H., Huang, X.: Intrusion detection system combined enhanced random forest with smote algorithm, pp. 1–29 (2021)
21. Tian, L., Lu, Y.: An intrusion detection model based on smote and convolutional neural network ensemble. J. Phys. Conf. Ser. **1828**(1), 1–10 (2021)
22. Vijayanand, R., Devaraj, D., Kannapiran, B.: A novel intrusion detection system for wireless mesh network with hybrid feature selection technique based on GA and MI. J. Intell. Fuzzy Syst **34**(3), 1243–1250 (2018)
23. Shafiq, M., Tian, Z., Bashir, A.K., Du, X., Guizani, M.: IoT malicious traffic identification using wrapper-based feature selection mechanisms. Comput. Secur. **94**, 101863 (2020)
24. Roopak, M., Tian, G.Y., Chambers, J.: Multi-objective based feature selection for ddos attack detection in iot networks. IET Netw. **9**(3), 120–127 (2020)
25. Siddiqi, M.A., Pak, W.: Optimizing filter-based feature selection method flow for intrusion detection system. Electronics **9**(12), 2114 (2020)
26. Ren, J., Guo, J., Qian, W., Yuan, H., Hao, X., Jingjing, H.: Building an effective intrusion detection system by using hybrid data optimization based on machine learning algorithms. Secur. Commun. Netw **2019**, 1–11 (2019)
27. Nimbalkar, P., Kshirsagar, D.: Feature selection for intrusion detection system in internet-of-things (IoT). ICT Express **7**(2), 177–181 (2021)
28. Elmasry, W., Akbulut, A., Zaim, A.H.: Evolving deep learning architectures for network intrusion detection using a double pso metaheuristic. Comput. Netw. **168**
29. Devan, P., Khare, N.: An efficient xgboost-dnn-based classification model for network intrusion detection system. Neural Comput. Appl. **32**(16), 12499–12514 (2020)
30. de Elias, E.M., et al.: A hybrid CNN-LSTM model for IIoT edge privacy-aware intrusion detection. In: IEEE Latin-American Conference on Communications (LATINCOM), Rio de Janeiro, Brazil, pp. 1–6 (2022). https://doi.org/10.1109/LATINCOM56090.2022.10000468
31. Javeed, D., Gao, T., Saeed, M.S., Kumar, P.: An intrusion detection system for edge-envisioned smart agriculture in extreme environment. IEEE Internet Things J. **11**(16), 26866–26876 (2024). https://doi.org/10.1109/JIOT.2023.3288544
32. Hakan, A., Zafer, A.: A hybrid CNN + LSTM-based intrusion detection system for industrial IoT networks. Eng. Sci. Technol. Int. J. **38** (2023). https://doi.org/10.1016/j.jestch.2022.101322
33. Moudoud, H., Khoukhi, L., Cherkaoui, S.: Prediction and detection of FDIA and DDoS attacks in 5G enabled IoT. IEEE Netw. **35**(2), 194–201 (2021). https://doi.org/10.1109/MNET.011.2000449
34. El Houda, Z.A., Moudoud, H., Brik, B., Khoukhi, L.: Securing federated learning through blockchain and explainable AI for robust intrusion detection in IoT networks. In: IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), pp. 1–6 (2023). https://doi.org/10.1109/INFOCOMWKSHPS57453.2023.10225769
35. Moudoud, H., Cherkaoui, S.: Empowering security and trust in 5G and beyond: a deep reinforcement learning approach. IEEE Open J. Commun. Soc. **4**, 2410–2420 (2023). https://doi.org/10.1109/OJCOMS.2023.3313352
36. El Houda, Z.A., Moudoud, H., Khoukhi, L.: Secure and efficient federated learning for robust intrusion detection in IoT networks. In: IEEE Global Communications Conference, pp. 2668–2673 (2023). https://doi.org/10.1109/GLOBECOM54140.2023.10436768
37. Singh, A., Prakash, J., Kumar, G.: Intrusion detection system: a comparative study of machine learning based IDS (2022)
38. Samed, A., Dener, M.: Stl-hdl: a new hybrid network intrusion detection system for imbalanced dataset on big data environment (2021)
39. Thiyam, B., Dey, S.: Efficient feature evaluation approach for a class-imbalanced dataset using machine learning. Proc. Comput. Sci. **218**, 2520–2532 (2023)
40. Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H.: Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and iiot applications for centralized and federated learning. IEEE Access Pract. Innov. Open Solut. **10**, 40281–40306 (2022)
41. CIC-DDoS2019 Dataset: Canadian Institute for Cybersecurity (2019). https://www.unb.ca/cic/datasets/ddos-2019.html. Accessed 6 Feb 2024
42. Filus, K., Boryszko, P., Domańska, J., Siavvas, M., Gelenbe, E.: Efficient feature selection for static analysis vulnerability prediction. Sensors **21**(4), 1133 (2021)
43. Khacha, A., Saadouni, R., Harbi, Y., Aliouat, Z.: Hybrid deep learning-based intrusion detection system for industrial internet of things. In: 5th International Symposium on Informatics and its Applications (ISIA), pp. 1–6. IEEE (2022)
44. Kunhare, N., Tiwari, R., Dhar, J.: Particle swarm optimization and feature selection for intrusion detection system. Sādhanā **45**(1), 1–14 (2020)
45. Tareq, I., Elbagoury, B.M., El-Regaily, S., El-Horbaty, E.-S.M.: Analysis of ToN-IoT, Unw-NB15, and Edge-IIoT datasets using Dl in cybersecurity for IoT. Appl. Sci. **12**(19), 9572 (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Mohemmed Yousuf Rahamathulla received the M.E. and B.E. degrees in CSE from Anna University. He is currently a Research Scholar with the Vellore Institute of Technology, Vellore, India. His research interests include Cybersecurity, IoT, and Machine Learning.



Mangayarkarasi Ramaiah received the Ph.D. degree in information technology and engineering from the Vellore Institute of Technology (VIT University), Vellore, India, and the M.E. degree in computer science from Anna University. She is currently an Associate Professor with the School of Computer Science Engineering and Information Systems, VIT University. She has attended many national and international conferences and published articles in reputed journals. Her research interests include computer vision, image processing, cyber security, machine learning, and artificial intelligence.